

Tunisian Republic
Ministry of Higher Education and Scientific Research



University of Tunis El Manar
Faculty of Economics and Management of
Tunis



FINAL YEAR PROJECT

Submitted in partial fulfillment of the requirements for the degree:
Bachelor's Degree in Business Intelligence

FYP Title

Host Organization
OneTech Business Solution



PREPARED BY
Mr Samaali Bassem

SUPERVISED BY
Professor Besma BEN AMARA
(Academic supervisor)
Mr Maher Daassi
(Professional supervisor)

ACADEMIC YEAR: 2025-2026

Acknowledgements

I would like to thank my academic supervisor, Mrs Besma BEN AMARA.

Thank you for your advice, your support, and your kind feedback during this project.

I also thank my professional supervisors, Mr Maher Daassi and Mrs Rahma Ouesleti, at
OneTech Business Solution.

Thank you for giving me this opportunity and for helping me step by step.

Their practical guidance helped me a lot during this work.

I thank all the professors and staff of the Faculty of Economic Sciences and Management of
Tunis.

I am also grateful to everyone who helped me, directly or indirectly, to finish this report.

Finally, I thank the jury members for taking the time to read and evaluate my work.

I hope this report is clear and meets their expectations.

Contents

Acknowledgements	1
Table of contents	4
List of abbreviations	5
General Introduction	6
1 General Framework of the Project	7
1.1 General Context of the Project	8
1.2 Company Presentation	8
1.2.1 Organizational Chart	8
1.2.2 Existing Information Systems	10
1.3 Problem Statement	11
1.3.1 Identified Pain Points	11
1.3.2 Project Objective	12
1.4 Requirements Analysis	12
1.4.1 Functional Requirements	12
1.4.2 Non-Functional Requirements	13
1.5 Development Methodology: Scrum	13
1.5.1 Why Scrum Was Chosen	13
1.5.2 Scrum Framework Overview	15
1.5.3 Sprint Plan	16
1.6 Chosen Technologies and Rationale	16
1.7 Technology Stack Table	17
1.8 Chapter Conclusion	18
2 System Analysis and Design	19
2.1 Introduction	20
2.2 Use Case Analysis	20
2.2.1 Actors and Access Scope	20
2.2.2 Global Use Case Diagram	21
2.2.3 Main Use Cases Description	22
2.3 Data Architecture and ETL-Oriented Design	22
2.3.1 Source Systems Exploration and Table Selection	22
2.3.2 View Layer for ETL Simplification	25
2.3.3 Unified PostgreSQL Target Schema and Integration Constraints	27

2.4	Global System Architecture	28
2.5	Chapter Conclusion	29
3	ETL and Unified Database	30
3.1	Introduction	31
3.2	Sprint Backlog	31
3.3	Sprint Design	31
3.3.1	Refinement of data rules and ETL mappings	31
3.4	Conception	32
3.4.1	ETL Pipeline Architecture	32
3.4.2	Data Validation and Control Flow	34
3.4.3	Key Design Advantages	35
3.5	Sprint Realization	35
3.5.1	Implementation outcomes	35
3.6	Conclusion	36
4	Authentication Module	37
4.1	Introduction	38
4.2	Sprint Backlog	38
4.3	Sprint Design	38
4.4	Conception	39
4.4.1	Use Case Refinement	39
4.4.2	Detailed Sequence Diagrams	41
4.5	Sprint Realization	44
4.5.1	Authentication Interface	44
4.5.2	Admin OTP Verification Interface	45
4.5.3	Registration Interface	45
4.5.4	Administrator Account Approval Interface	46
4.5.5	Approval Email Notification	46
4.5.6	Error Handling Scenarios	47
4.6	Security Considerations	47
4.7	Conclusion	48
5	Administration Module	49
5.1	Introduction	50
5.2	Sprint Backlog	50
5.3	Sprint Design	50
5.3.1	Dashboard module design	50
5.3.2	Projects module design	51
5.4	Conception	51
5.4.1	Administration use case diagram	51
5.5	Sprint Realization	52
5.5.1	Implementation outcomes	52
5.5.2	Administration dashboard and users interface	53
5.5.3	Projects interface	53
5.5.4	Assign project interface	54
5.5.5	Settings page interface	54
5.5.6	Filtering and pagination interface	55

5.6	Conclusion	55
6	HR Functional Modules	56
6.1	Introduction	57
6.2	Sprint Backlog	57
6.3	Sprint Design	57
6.3.1	Employees page design mockup	57
6.3.2	Check-In/Out page design mockup	58
6.3.3	Employee Leave page design mockup	58
6.4	Conception	59
6.4.1	HR use case diagram	59
6.5	Sprint Realization	60
6.5.1	Implementation outcomes	60
6.5.2	Employees page interface	61
6.5.3	Check-In/Out page interface	61
6.5.4	Employee Leave page interface	62
6.6	Conclusion	62
7	Project Manager Modules	63
7.1	Introduction	64
7.2	Sprint Backlog	64
7.3	Sprint Design	64
7.3.1	Projects page design mockup	64
7.3.2	Members page design mockup	65
7.3.3	Members Attendance page design mockup	65
7.4	Conception	66
7.4.1	Project Manager use case diagram	66
7.5	Sprint Realization	67
7.5.1	Implementation outcomes	67
7.5.2	Projects page interface	68
7.5.3	Members page interface	68
7.5.4	Members attendance monitoring interface	69
7.6	Conclusion	69
8	RH Agent Integration	70
8.1	Introduction	71
8.2	Sprint Backlog	71
8.3	Sprint Design	71
8.3.1	Conversation and interaction design	71
8.3.2	LangGraph routing design	72
8.4	Conception	73
8.4.1	RH Agent sequence diagram	73
8.4.2	Agent state model conception	74
8.5	Sprint Realization	74
8.5.1	Implementation outcomes	74
8.5.2	RH Agent interface	75
8.6	Conclusion	75

List of abbreviations

- AB: Administration Backlog
- AI: Artificial Intelligence
- API: Application Programming Interface
- CSS: Cascading Style Sheets
- ETL: Extract, Transform, Load
- FR: Functional Requirement
- FYP: Final Year Project
- GitHub: Git-based source code hosting platform
- HR: Human Resources
- HTTP: Hypertext Transfer Protocol
- ICT: Information and Communication Technology
- IDE: Integrated Development Environment
- ID: Identifier
- IT: Information Technology
- JWT: JSON Web Token
- KPI: Key Performance Indicator
- LLM: Large Language Model
- NFR: Non-Functional Requirement
- ORM: Object-Relational Mapping
- PDF: Portable Document Format
- PM: Project Manager
- RBAC: Role-Based Access Control
- REST: Representational State Transfer
- RH: Ressources Humaines
- SQL: Structured Query Language
- UI: User Interface

General Introduction

This final year project was carried out at OneTech Business Solutions (OTBS) as part of the requirements for obtaining the Bachelor's Degree in Business Intelligence. It was completed in a professional environment where attendance monitoring, HR follow-up, and project management rely on clear, reliable, and well-structured information systems.

The starting point of this work was a real operational need within the company. OTBS already relied on two separate platforms: **BioTime**, which was used for biometric attendance tracking, and **EasyProject**, which was used for HR and project management. Although each platform was useful within its own scope, they were not directly connected. As a result, attendance follow-up was not always consistent, especially when employees were working on external missions and therefore appeared active in projects without appearing in office attendance records.

To address this issue, the project aimed to design and implement a unified solution capable of integrating data from both systems into a centralized database and a dedicated web application. The proposed platform was designed to support several user roles, including administrators, HR staff, and project managers, while also ensuring secure access, data consistency, and easier reporting.

From a technical perspective, this work covered the main stages of a complete full-stack project. It included the analysis of existing systems, the design of the target architecture, the implementation of ETL pipelines, the development of a FastAPI backend, the creation of a React frontend, and the integration of role-based authentication with dedicated business modules. The project also explored the integration of an RH Agent in order to improve user interaction and support daily operational tasks.

This report presents the work carried out throughout the internship. It begins with the general framework of the project, then moves to the analysis and design phase, the ETL and unified database implementation, the authentication and administration modules, the HR and Project Manager features, and finally the integration of the RH Agent.

Chapter 1

General Framework of the Project

Contents

1.1	General Context of the Project	8
1.2	Company Presentation	8
1.2.1	Organizational Chart	8
1.2.2	Existing Information Systems	10
1.3	Problem Statement	11
1.3.1	Identified Pain Points	11
1.3.2	Project Objective	12
1.4	Requirements Analysis	12
1.4.1	Functional Requirements	12
1.4.2	Non-Functional Requirements	13
1.5	Development Methodology: Scrum	13
1.5.1	Why Scrum Was Chosen	13
1.5.2	Scrum Framework Overview	15
1.5.3	Sprint Plan	16
1.6	Chosen Technologies and Rationale	16
1.7	Technology Stack Table	17
1.8	Chapter Conclusion	18

1.1 General Context of the Project

This chapter introduces the organizational context of the internship, explains the problem addressed by the project, presents the main system requirements, and justifies the development methodology that was adopted.

1.2 Company Presentation

This internship was carried out at **OneTech Business Solutions (OTBS)**. OTBS is part of the OneTech Group and belongs to its ICT activity area [1, 2]. The company specializes in technology integration and IT solutions for business clients. Since several teams often work on different projects at the same time, day-to-day coordination depends heavily on reliable digital tools.

Within the HR and project follow-up workflow, two internal systems were already in use: **BioTime** for attendance tracking and **EasyProject** for HR and project-related data. Although each system functioned correctly on its own, they were not directly synchronized.

Company	OneTech Business Solutions (OTBS)
Industry	ICT / IT integration services
Group Affiliation	Subsidiary of OneTech Group (ICT activity)
Business Focus	Integration and distribution of information and communication technologies (ICT)
Existing Systems	BioTime (ZKTeco biometric attendance) + EasyProject (project & HR management)
Infrastructure	On-premise PostgreSQL and MySQL databases, internal network
Internship Duration	3 months (approximately 12 weeks)
Intern Role	Full-Stack Developer — Design, ETL, Backend (FastAPI), Frontend (React)

1.2.1 Organizational Chart

The organizational chart in Figure 1.1 presents the group management structure of the company and highlights its position within the broader organizational hierarchy.

1.2.2 Existing Information Systems

At the time of the internship, the company relied on two separate information systems that supported different operational needs. Each system was useful in its own scope, but each one also represented only part of the information needed for complete attendance follow-up.

BioTime was the attendance-oriented system used in connection with the company’s biometric device infrastructure. Its main role was to capture employee presence events such as check-in and check-out actions, together with timestamps and employee identifiers. In practice, this system provided the most direct view of physical presence at the office. From an HR perspective, BioTime was important because it produced the raw operational evidence used to verify punctuality, presence, and attendance history. However, by design, it focused mainly on biometric attendance events and did not explain why an employee might be absent from the office while still being officially active on external work.

Figure 1.2 is reserved for an illustration of the BioTime software interface or architecture.

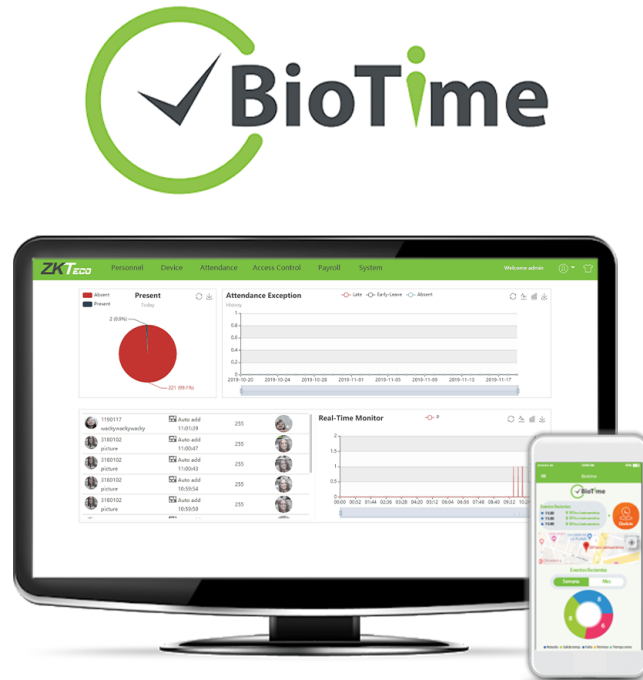


Figure 1.2: Placeholder for BioTime software presentation

EasyProject was the project and HR-oriented system used to manage organizational and operational data beyond simple attendance events. It stored information related to employees, project assignments, organizational structure, schedules, and leave or mission context depending on the business workflow. This made EasyProject valuable for understanding the administrative and project-related explanation behind an employee’s status. For example, when an employee was assigned to a mission, linked to a project, or covered by leave information, EasyProject could provide the context that BioTime alone could not show. As a result, it played a complementary role by describing the business meaning behind employee activity.

Figure 1.3 is reserved for an illustration of the EasyProject software interface or architecture.



Figure 1.3: Placeholder for EasyProject software presentation

In practice, these systems operated in isolation. BioTime described physical attendance, while EasyProject described organizational and project context. Since there was no native integration between them, overall visibility remained limited and data reconciliation had to be done manually.

1.3 Problem Statement

1.3.1 Identified Pain Points

Before this project began, the company already had two systems related to attendance follow-up, but each one reflected only part of the real situation.

BioTime is connected to the ZKTeco device installed at the company. It records clock-in and clock-out actions when employees are physically present at the office. **EasyProject**, on the other hand, is used for project follow-up and stores project-related information such as team members, assignments, and mission orders.

The main problem appeared when an employee was not physically present at the company because they were on an outside mission with a client. In such a case, the employee could be correctly registered in EasyProject as working on a specific mission or project, while no clock-in or clock-out record appeared in BioTime because the employee had not passed through the company device. For the HR department, this created a confusing situation: the employee seemed absent in BioTime but active in EasyProject.

Because of this separation, the company faced several operational difficulties:

- Attendance data was scattered between BioTime and EasyProject.
- The HR department had no unified view of the actual attendance status of employees.
- An employee could appear absent in one system while being active on a mission in the other.
- Reporting required manual verification and comparison between both systems.
- Manual reporting consumed time and increased the risk of errors.
- Managers and HR staff lacked a centralized tool for follow-up and decision-making.

1.3.2 Project Objective

The main objective of this internship project was to design a solution capable of bringing together BioTime and EasyProject within a centralized database and a unified application, mainly to simplify reporting for the HR department.

More specifically, the project aimed to:

- collect attendance data from BioTime and project-related mission data from EasyProject;
- integrate both data sources into a centralized PostgreSQL database through an ETL process;
- reconcile differences between physical office attendance and project mission activity;
- provide the HR department with a clearer and more reliable view of employee attendance;
- reduce manual reporting efforts and make follow-up faster and easier;
- provide secure, role-based access through a web application for Admin, HR, and Project Manager users.

1.4 Requirements Analysis

1.4.1 Functional Requirements

Administrator

- **FR-01:** Create and manage user accounts.
- **FR-02:** Assign roles (Admin, HR, Project Manager).
- **FR-03:** Assign projects to Project Managers.
- **FR-04:** Assign Project Managers to projects.
- **FR-05:** The Administrator shall not access employee details or attendance records.

Human Resources (HR)

- **FR-06:** View all employees across the organization.
- **FR-07:** View attendance records for all employees.
- **FR-08:** View attendance summaries and reports at global scope.
- **FR-09:** Manage HR attendance and leave follow-up operations.

Project Manager

- **FR-10:** View only projects assigned by the Administrator.
- **FR-11:** View members belonging to assigned projects.
- **FR-12:** View attendance records of members in assigned projects only.

ETL & System

- **FR-13:** Extract attendance and HR data from BioTime and EasyProject on schedule.
- **FR-14:** Resolve duplicate records, ID mismatches, and timezone differences.
- **FR-15:** Store transformed and unified data in the main PostgreSQL database through SQLAlchemy.

- **FR-16:** Provide real-time synchronization capabilities.

Dashboard & Reporting

- The dashboard shall display KPIs (present employees, absences, late arrivals, overtime).
- The system shall allow filtering by date range, project, and employee.
- Reports shall be exportable in standard formats.

1.4.2 Non-Functional Requirements

Category	Requirement ID	Description
Performance	NFR-01	API endpoints shall respond within 300ms for standard queries on datasets up to 5,000 employees.
Security	NFR-02	All API routes shall be protected by JWT-based authentication. Passwords shall be hashed using bcrypt.
Security	NFR-03	Role-based access control (RBAC) shall enforce permission boundaries.
Reliability	NFR-04	ETL shall handle source failures, log errors, and resume from the last successful sync point.
Data Integrity	NFR-05	Duplicate attendance records shall be prevented through ETL deduplication and unique constraints.
Scalability	NFR-06	The architecture shall support additional devices and new employee data sources.
Usability	NFR-07	The React interface shall be responsive on desktop and tablet screens.
Maintainability	NFR-08	Database migrations shall be managed with Alembic.
Portability	NFR-09	The application shall be Docker-compatible for deployment flexibility.

1.5 Development Methodology: Scrum

1.5.1 Why Scrum Was Chosen

Choosing the right development methodology was important for this project because the internship was short and the work included several parts at the same time. The project was not only about building one interface or one database. It also included system analysis, ETL design, backend development, frontend implementation, authentication, and RH Agent integration. Because of that, we needed a method that was organized but still flexible enough to adapt when needed.

For this reason, **Scrum** was the most suitable choice. It gave us a clear way to organize the work in small steps while still allowing changes and improvements during the internship.

(a) Requirements were not fully clear from the start

At the beginning of the project, the main objective was clear, but some details were still unknown. For example, the exact logic needed to reconcile BioTime attendance data with EasyProject mission data became clearer only after exploring the databases and testing different cases. In this kind of situation, Scrum is useful because it allows the solution to improve step by step instead of forcing everything to be fixed from the beginning.

(b) The internship duration was limited

The internship lasted about three months, so time management was very important. Scrum helped divide the work into short sprints, and each sprint produced a visible result. This made the progress easier to follow and helped keep the project moving in a structured way.

(c) The project was divided into several parts

The project naturally included several layers such as data exploration, ETL pipelines, database design, backend APIs, frontend modules, and agent integration. Scrum fit this structure well because each sprint could focus on one important part of the system while keeping the whole project connected.

(d) Technical problems needed iterative work

During the project, several technical issues appeared, such as missing identifiers, duplicated data, timestamp differences, and integration constraints between the two source systems. These issues could not always be solved directly in one attempt. Scrum was helpful because it allowed us to test, review, and improve the work from one sprint to another.

(e) Scrum vs. Waterfall

Waterfall is a more linear methodology. It usually starts with complete requirements, then moves step by step through analysis, design, implementation, and testing. This approach works well when the requirements are stable and well known from the beginning.

In our case, that was not really possible. Some requirements became clearer only during the work, especially after studying the source systems and starting implementation. If we had followed Waterfall, any change discovered later would have made the process heavier and less flexible.

Compared with Waterfall, Scrum was better for this project because:

- the requirements were still evolving;
- the project needed regular progress during a short internship;
- technical issues required continuous adjustment;
- working in small increments was more practical than following one rigid sequence.

(f) Scrum vs. Kanban

Kanban is also an agile method, but it focuses more on continuous task flow than on fixed sprints. It is very useful when work arrives continuously, like support or maintenance tasks.

In this project, the work followed a clearer progression from analysis to implementation, then to advanced modules and integration. Because of that, Scrum was more suitable than Kanban

alone. Scrum gave us fixed periods with clear goals, which made it easier to organize the internship work and track results.

Compared with Kanban, Scrum was more adapted here because:

- it provided clear sprint objectives;
- it made milestone tracking easier;
- it helped structure the internship around deliverable results;
- it still kept enough flexibility to adjust the work when needed.

(g) Final choice

In the end, Scrum was chosen because it gave the best balance between structure and flexibility. It was organized enough to manage the project during a limited internship period, but also flexible enough to adapt to technical discoveries and requirement changes. That is why it was the most appropriate methodology for this project.

1.5.2 Scrum Framework Overview

Roles

- Product Owner: company supervisor.
- Scrum Master / Developer: intern.

Artifacts

- Product Backlog.
- Sprint Backlog.
- Increment.

Ceremonies

- Sprint Planning.
- Sprint Review.
- Sprint Retrospective.

1.5.3 Sprint Plan

Sprint	Duration	Main Objective
Sprint 1	Week 1 – Week 2	Chapter 2 — System analysis and design: study the source systems, define the required data scope, and prepare the main conception artifacts.
Sprint 2	Week 3 – Week 4	Chapter 3 — ETL and unified database: implement the ETL pipelines and populate the centralized PostgreSQL database.
Sprint 3	Week 5 – Week 6	Chapter 4 — Authentication module: implement login, registration, administrator approval, and role-based access control.
Sprint 4	Week 7 – Week 8	Chapter 5 — Administration module: deliver user management, project assignment, and synchronization settings features.
Sprint 5	Week 9 – Week 10	Chapter 6 — HR functional modules: implement employee management, attendance follow-up, and leave handling interfaces.
Sprint 6	Week 11 – Week 12	Chapter 7 — Project Manager modules: deliver project, member, and attendance monitoring features for project managers.
Sprint 7	Week 13 – Week 14	Chapter 8 — RH Agent integration: integrate the RH Agent and finalize the intelligent assistance workflow.

1.6 Chosen Technologies and Rationale

To solve the integration problem between different systems while still building a maintainable full-stack solution, a focused technology stack was selected. This choice was guided by compatibility, reliability, development speed, and long-term maintainability.

1.7 Technology Stack Table

Icon	Technology	Description	Why I Chose It
	SQLAlchemy	Python ORM and SQL toolkit used in ETL and backend data access.	One clear way to work with both PostgreSQL and MySQL, which reduces complexity and repeated logic.
	FastAPI	Framework used to build REST APIs with validation and automatic documentation.	It allowed rapid API development with a clean structure and good performance.
ALEMBIC	Alembic	Tool used to manage and track database schema changes.	It keeps database changes clear, safe, and easy to track.
	PostgreSQL	Main unified relational database for transformed attendance and HR data.	Reliable, stable, and well suited for both daily operations and structured reporting.
	Redis	In-memory data store used to speed up temporary data access and support fast backend operations.	Useful when low-latency access is needed for cached or short-lived data.
	MySQL (Source)	Source database used by EasyProject and read by ETL jobs.	It let us integrate existing company data without changing the legacy system.
	Resend	Email delivery service for notifications and account communication.	Simple to use and reliable for sending account and notification emails.
	Docker	Platform used to package backend services and their dependencies.	It makes setup easier and keeps environments consistent.
	React	Frontend library used to build dynamic interfaces with reusable components.	It made the interface easier to organize and reuse.

Icon	Technology	Description	Why I Chose It
	Tailwind CSS	CSS framework used to style the interface quickly.	It helped us build pages faster with a more consistent design.
	shadcn/ui	Reusable UI components built on top of Radix and Tailwind.	It helped us create clean interfaces faster while keeping flexibility.
	TanStack Table	Table library used for sorting, filtering, and pagination.	It was very useful for large employee and attendance tables.
	TanStack Query	Library used for API state management and caching.	It improved performance and made async data handling cleaner.
	LangGraph	Framework used to build stateful, multi-step LLM workflows and agent graphs.	It was useful for organizing the RH Agent flow in a controlled way.
	LangChain	Framework used for prompt chaining, tool use, and retrieval integration.	It made AI feature integration faster and easier.
	Ollama	Local runtime used to serve open-source language models on private infrastructure.	It allowed private AI inference that fits company data-security needs.
	VS Code	Lightweight and extensible editor used for frontend and backend development.	It gave a fast editing experience with many useful extensions.
	PyCharm	Python IDE used for backend and ETL development.	It made Python development easier with good debugging and code tools.
	Git	Version control system used to track and manage source code changes.	It made collaboration and sprint work safer and easier to manage.
	GitHub	Remote repository hosting and collaboration platform built around Git.	It helped the team share code, review changes, and stay organized.

Overall, this stack helped address the main project challenge: connecting two separate source systems, exposing secure and maintainable APIs, and building a responsive interface for day-to-day follow-up.

1.8 Chapter Conclusion

This chapter established the foundation of the project by presenting the organizational context and the main challenge created by the coexistence of two isolated systems: BioTime and EasyProject.

The proposed response is a unified architecture built around an ETL pipeline, a consolidated PostgreSQL database, a FastAPI backend, and a React web dashboard designed to ensure reliable and conflict-free attendance monitoring.

Chapter 2

System Analysis and Design

Contents

2.1	Introduction	20
2.2	Use Case Analysis	20
2.2.1	Actors and Access Scope	20
2.2.2	Global Use Case Diagram	21
2.2.3	Main Use Cases Description	22
2.3	Data Architecture and ETL-Oriented Design	22
2.3.1	Source Systems Exploration and Table Selection	22
2.3.2	View Layer for ETL Simplification	25
2.3.3	Unified PostgreSQL Target Schema and Integration Constraints	27
2.4	Global System Architecture	28
2.5	Chapter Conclusion	29

2.1 Introduction

In this chapter, we explain the analysis and design work done before coding. We describe who uses the system, how they interact with it, how data is organized, and what global architecture was chosen.

2.2 Use Case Analysis

2.2.1 Actors and Access Scope

The platform has three main actors. Each actor has a clear role in the system:

- **Administrator (Admin)**: manages accounts and project assignment. The Admin creates accounts, approves accounts, and assigns projects to Project Managers.
- **Human Resources (HR)**: works on company-wide HR follow-up. HR can view employees, employees' clock-in/out records, and employees' leave information.
- **Project Manager**: works at team level. The Project Manager can view members, assigned projects, and team attendance.

2.2.2 Global Use Case Diagram

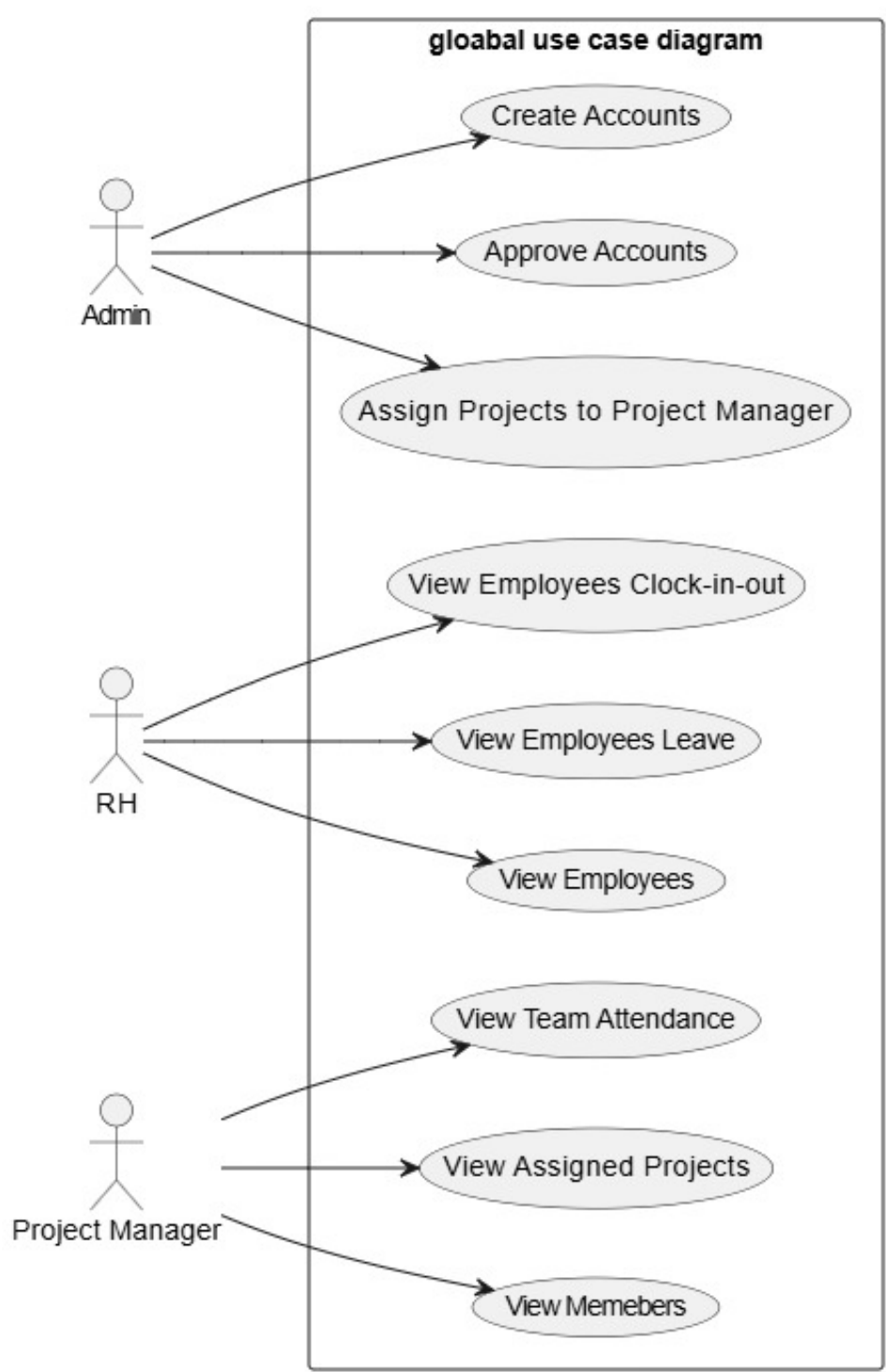


Figure 2.1: Global Use Case Diagram

Diagram logic (actor – use case mapping):

- **Admin:** create accounts, approve accounts, assign projects to Project Managers.
- **HR:** view employees, view employees' clock-in/out, view employees' leave.
- **Project Manager:** view members, view assigned projects, view team attendance.

2.2.3 Main Use Cases Description

Actor	Use Case	Description
Administrator	Create Accounts	Creates HR and Project Manager accounts so users can access the platform with the right role.
Administrator	Approve Accounts	Reviews pending accounts and approves them before users can log in.
Administrator	Assign Projects to Project Manager	Assigns one or more projects to a Project Manager.
Human Resources	View Employees	Opens the employee list and checks employee information across the company.
Human Resources	View Employees Clock-in/out	Checks employee attendance records for follow-up and daily monitoring.
Human Resources	View Employees Leave	Reviews leave information to understand absences and missing attendance actions.
Project Manager	View Members	Sees the members linked to the projects under their responsibility.
Project Manager	View Assigned Projects	Sees only the projects assigned to them by the Admin.
Project Manager	View Team Attendance	Monitors the attendance of team members in assigned projects.

2.3 Data Architecture and ETL-Oriented Design

2.3.1 Source Systems Exploration and Table Selection

The two source systems, **BioTime** and **EasyProject**, contain many operational tables. To keep the integration clear and efficient, we first explored both schemas and kept only the entities needed for ETL.

1) Selected BioTime Tables Figure 2.2 presents the subset of tables selected from BioTime after analyzing a source schema containing roughly 200 tables. The selected tables correspond to the entities strictly required for attendance extraction, identity matching, and time-event tracking. Figure 2.2 shows the BioTime tables kept after the selection phase. These tables are the ones needed for attendance extraction, identity matching, and event tracking.

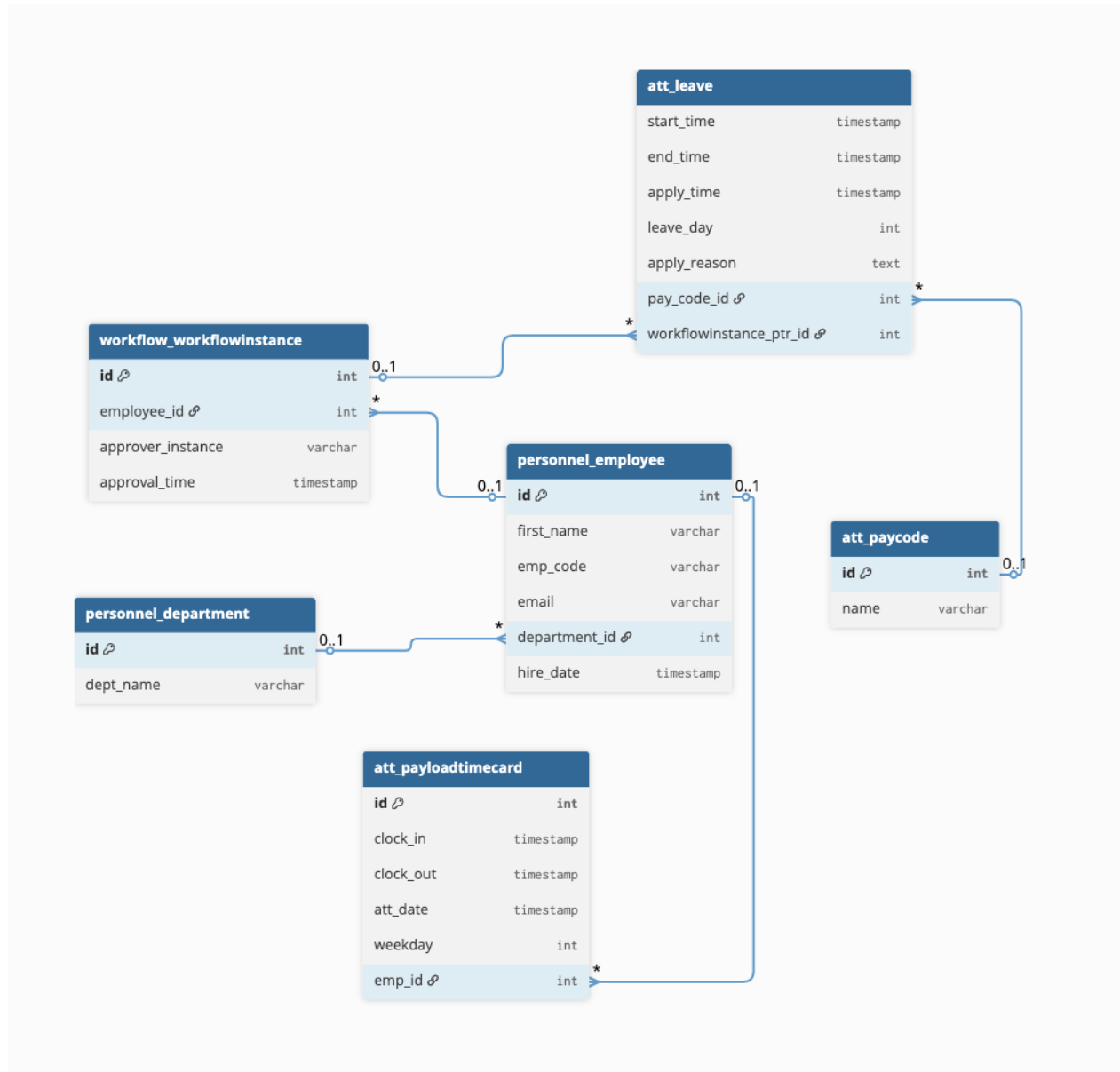


Figure 2.2: Selected Tables from BioTime

2) Selected EasyProject Tables Figure 2.3 shows the EasyProject tables kept for integration. The focus was on employee data, organization structure, project assignment, and leave information used during transformation.

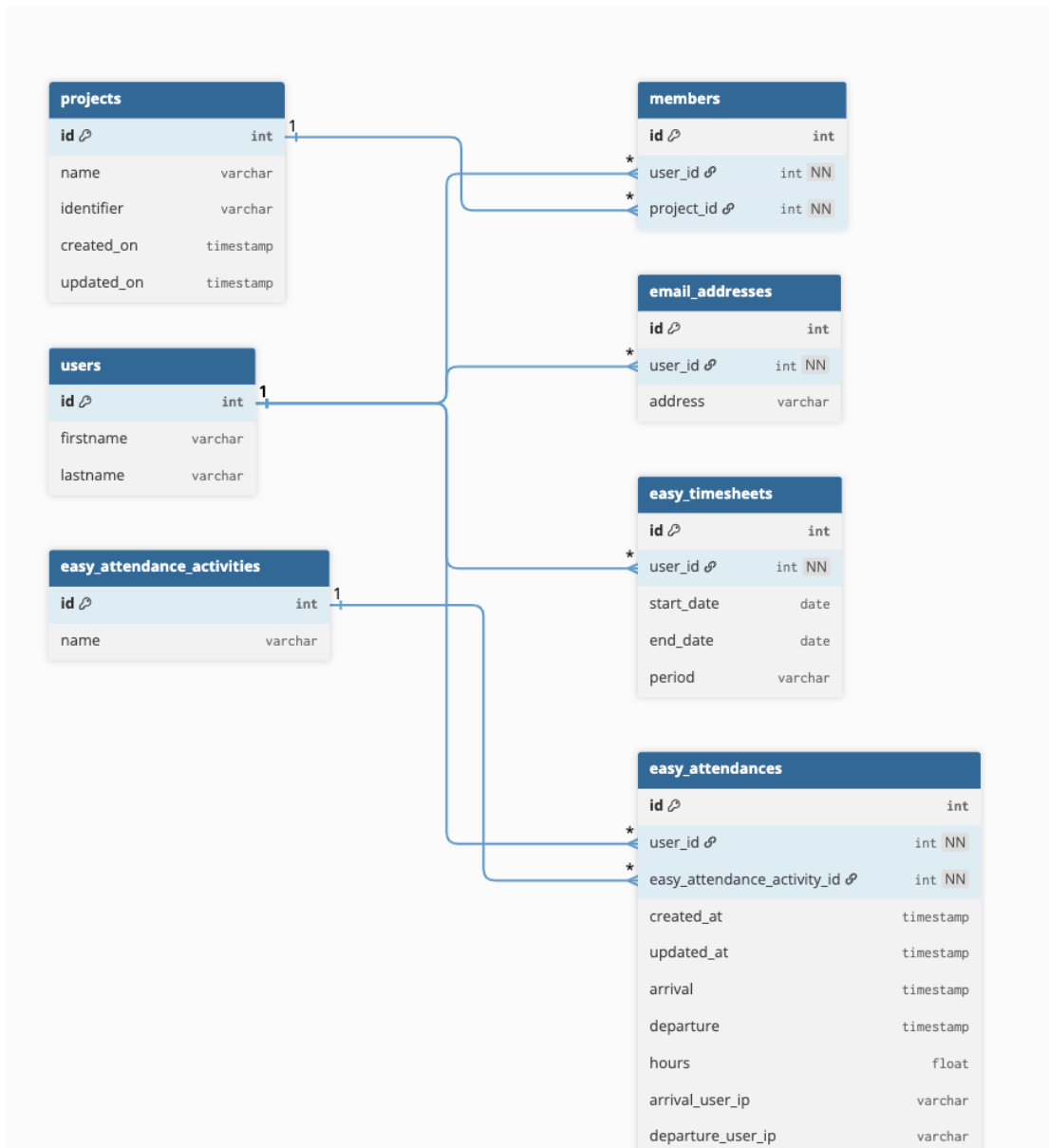


Figure 2.3: Selected Tables from EasyProject

2.3.2 View Layer for ETL Simplification

After selecting the source tables, we created SQL views in each system. These views standardize field names, reduce join complexity, and provide clean ETL inputs.

3) BioTime Views Created After Table Selection Figure 2.4 illustrates the BioTime view layer. It isolates useful attendance data and gives a stable extraction interface for ETL.

biotime_employee_view		check_in_out_view		biotime_att_leave_view	
ep_emp_id	int	id	int	start_time	timestamp
emp_fullname	varchar	clock_in	timestamp	end_time	timestamp
emp_code	varchar	clock_out	timestamp	apply_time	timestamp
email	varchar	att_date	timestamp	leave_day	int
department_id	int	weekday	int	apply_reason	text
dept_name	varchar	emp_id	int	pay_code_id	int
hire_date	timestamp	email	varchar	name	varchar
		first_name	varchar	emp_id	int
		department_id	int	approver_instance	varchar
		dept_name	varchar	approval_time	timestamp
				email	varchar
				first_name	varchar
				dept_name	varchar

Figure 2.4: Views Created from BioTime Selected Tables

4) EasyProject Views Created After Table Selection Figure 2.5 presents the EasyProject views. This intermediate layer simplifies access to HR and project attributes and harmonizes data before loading.

easy_employee_view	
ep_emp_id	int
emp_fullname	varchar
email	varchar

easy_attendance_full_view	
id	int
created_at	timestamp
updated_at	timestamp
arrival	timestamp
departure	timestamp
hours	float
arrival_user_ip	varchar
departure_user_ip	varchar
ep_emp_id	int
emp_fullname	varchar
email	varchar
activity_id	int
activity_name	varchar

Figure 2.5: Views Created from EasyProject Selected Tables

2.3.3 Unified PostgreSQL Target Schema and Integration Constraints

The transformed outputs from both view layers are loaded into one unified PostgreSQL schema. At this stage, identifiers and timestamps are normalized, duplicates are removed, and integrity constraints are applied to keep analytics consistent across systems.

Main Application Diagram (Global Architecture Placeholder) Figure 2.6 summarizes the main structure of the application. It shows how data moves from source systems to ETL, then to the unified database, backend services, and user modules.

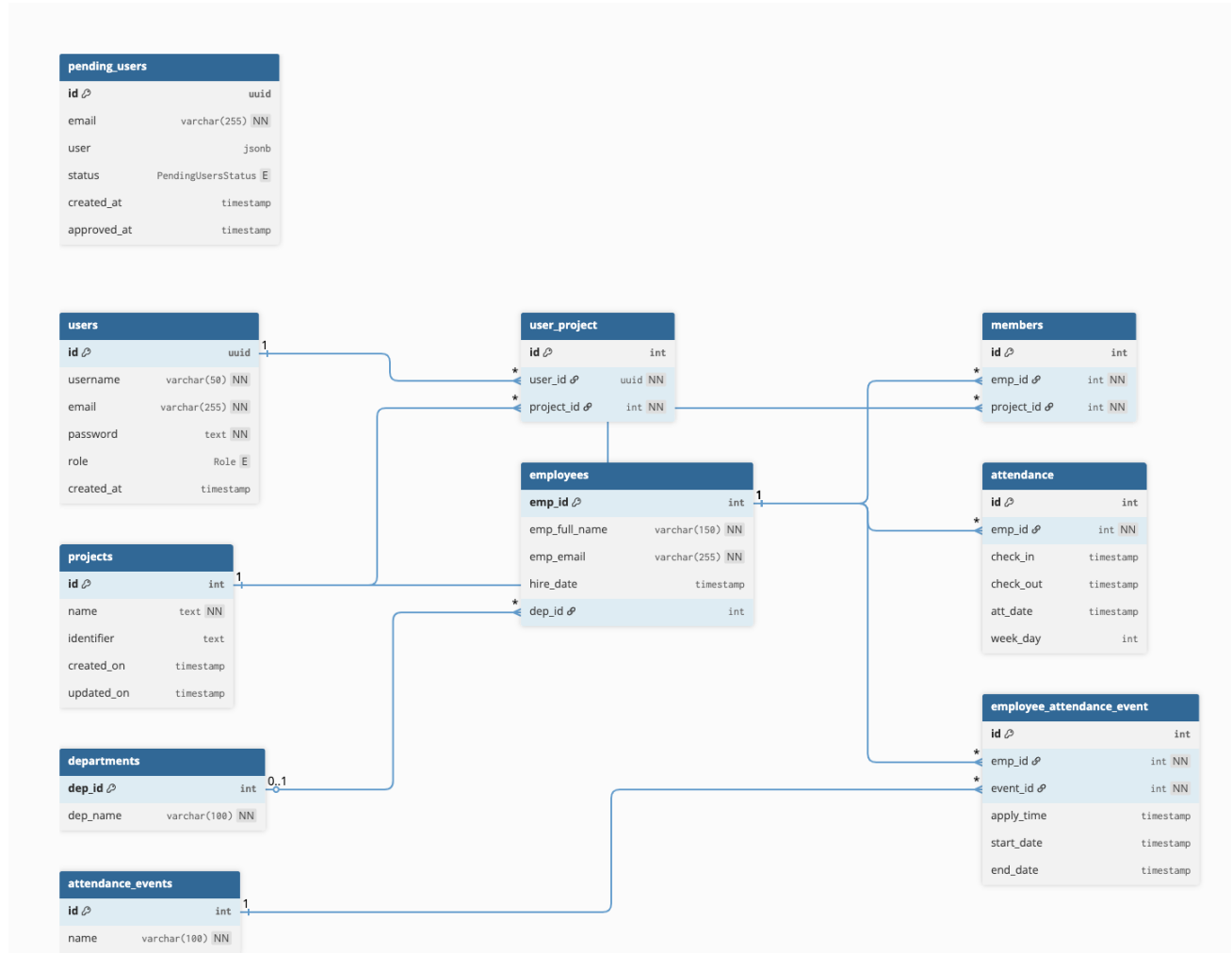


Figure 2.6: Main Application Diagram

2.4 Global System Architecture

The architecture starts by extracting data from **BioTime (PostgreSQL)** and **EasyProject (MySQL)** with **SQLAlchemy**. The data is then transformed and loaded into the main application database.

On top of this unified database, the backend uses **FastAPI**, **Alembic**, and **SQLAlchemy**. It exposes endpoints consumed by a frontend built with **React**, **Tailwind CSS**, and **shadcn/ui**.

Global System Architecture Diagram Figure 2.7 presents the end-to-end architecture, from source systems and ETL processing to the unified database, backend services, and frontend modules.

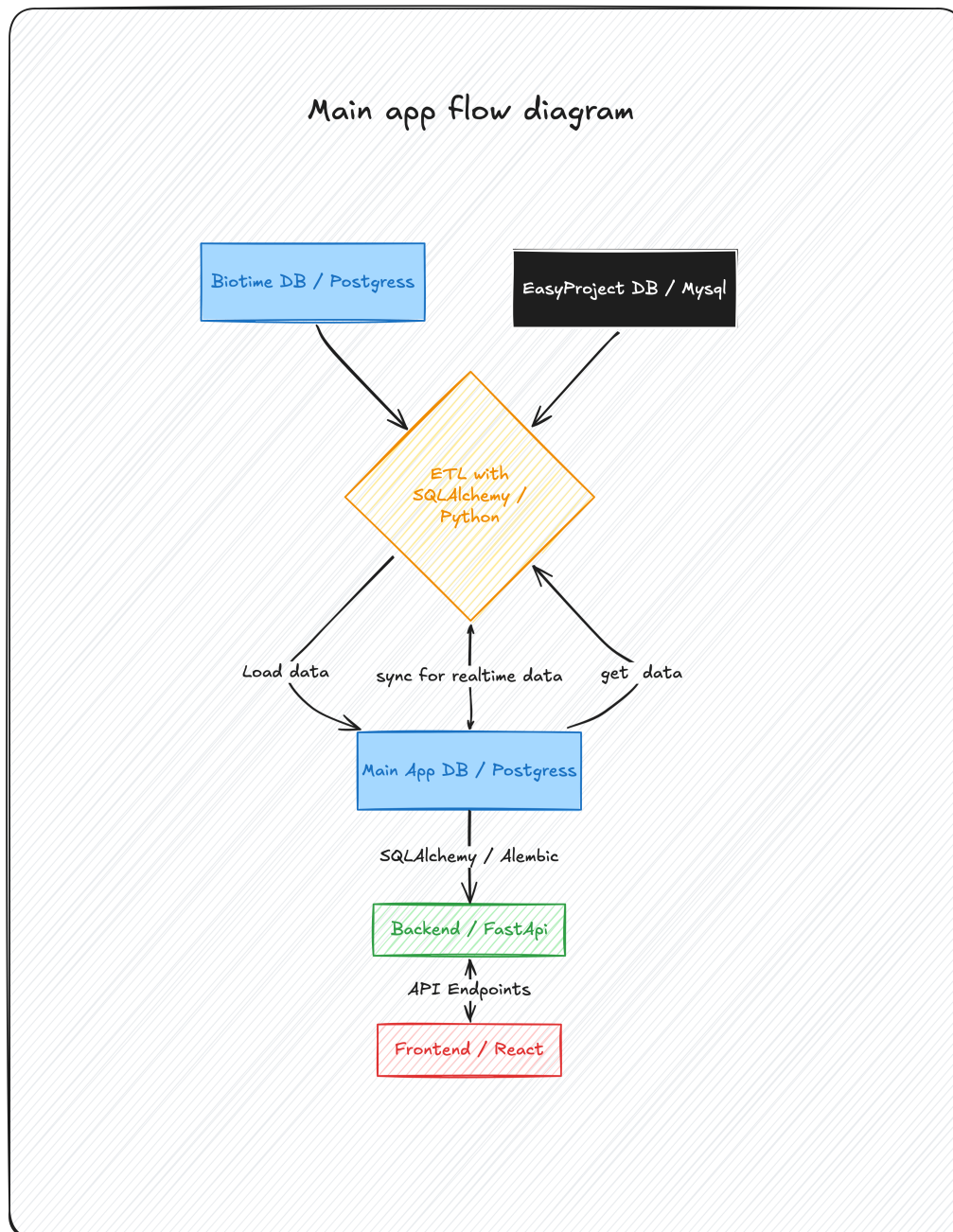


Figure 2.7: Global System Architecture Flow

2.5 Chapter Conclusion

This chapter explained the analysis and design choices of the solution. It covered actors and use cases, ETL-focused data architecture decisions, and the global system architecture. The next chapter focuses on authentication architecture and workflows.

Chapter 3

ETL and Unified Database

Contents

3.1	Introduction	31
3.2	Sprint Backlog	31
3.3	Sprint Design	31
3.3.1	Refinement of data rules and ETL mappings	31
3.4	Conception	32
3.4.1	ETL Pipeline Architecture	32
3.4.2	Data Validation and Control Flow	34
3.4.3	Key Design Advantages	35
3.5	Sprint Realization	35
3.5.1	Implementation outcomes	35
3.6	Conclusion	36

3.1 Introduction

This chapter presents Sprint 3, focused on building the ETL foundation and the unified operational database. The goal was to bring data from different systems into one clean, consistent, and monitorable flow before using it in business modules.

3.2 Sprint Backlog

The sprint backlog for Sprint 3 was formalized as user stories focused on ETL and unified database delivery:

ID	User Story	Priority
US1	As a system administrator, I want to connect to BioTime and EasyProject data sources so that ETL can extract operational data.	High
US2	As a data engineer, I want to build entity-based extraction pipelines (departments, employees, attendance, projects, members) to structure ingestion.	High
US3	As a data engineer, I want to apply transformation and normalization rules so that unified data remains clean and consistent.	High
US4	As a system, I want to load transformed records into the PostgreSQL target schema while preserving referential integrity.	High
US5	As an administrator, I want to monitor ETL execution status and errors in real time so that failures can be detected early.	Medium

3.3 Sprint Design

Sprint design followed an ETL-first approach. We apply integrity and control rules before application services consume the data.

3.3.1 Refinement of data rules and ETL mappings

At this stage, we refined the mapping rules between BioTime and EasyProject so the data could be loaded in a clean and consistent way. We aligned identifiers, normalized data types, and handled common conflicts between the two sources. We also paid special attention to duplicates, attendance consistency, and the loading order between related entities.

3.4 Conception

3.4.1 ETL Pipeline Architecture

The ETL process uses a modular and scalable architecture. This design improves maintainability, traceability, and consistency in the unified database.

1. Data Sources

Data is extracted from two main external systems:

- **BioTime**: attendance and employee activity data.
- **EasyProject**: project and member-related data.

The extraction layer uses **SQLAlchemy (Async ORM)** for efficient asynchronous communication with source databases.

2. Modular Pipeline Design

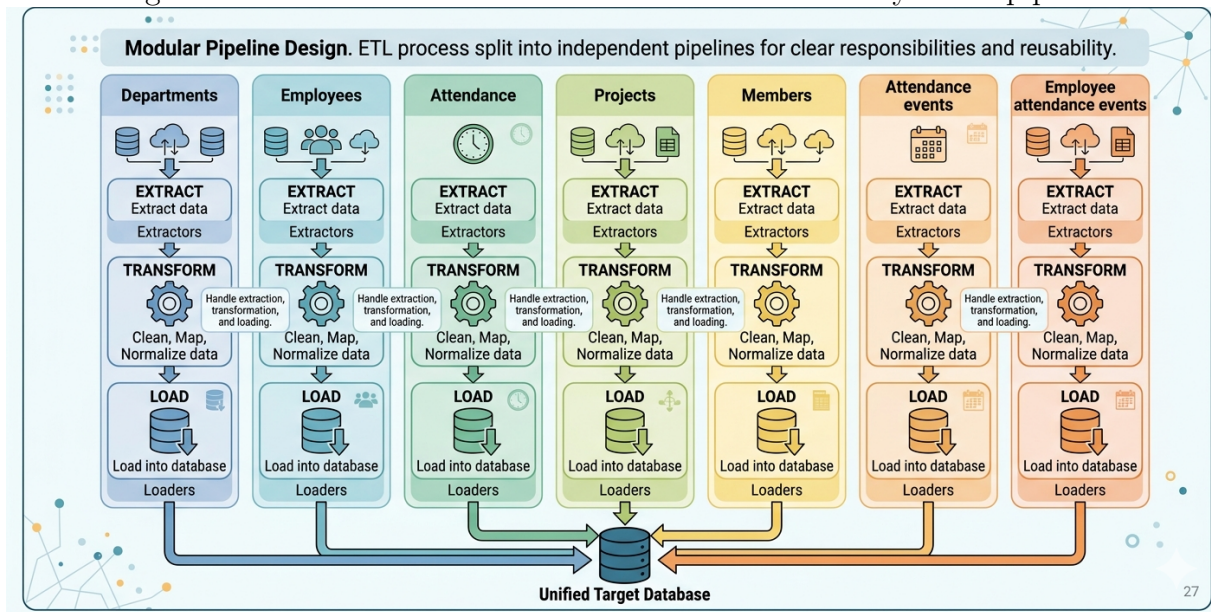
To keep responsibilities clear and reusable, the ETL process is split into independent pipelines. Each pipeline handles one entity:

- Departments pipeline
- Employees pipeline
- Attendance pipeline
- Projects pipeline
- Members pipeline
- Attendance events pipeline
- Employee attendance events pipeline

Each pipeline handles extraction, transformation (cleaning, mapping, normalization), and loading into the unified target database.

Figure 3.1 shows how independent pipeline modules separate extraction, transformation, and loading responsibilities.

Figure 3.1: Modular ETL architecture built around entity-based pipelines



3. Orchestration via Main Pipeline

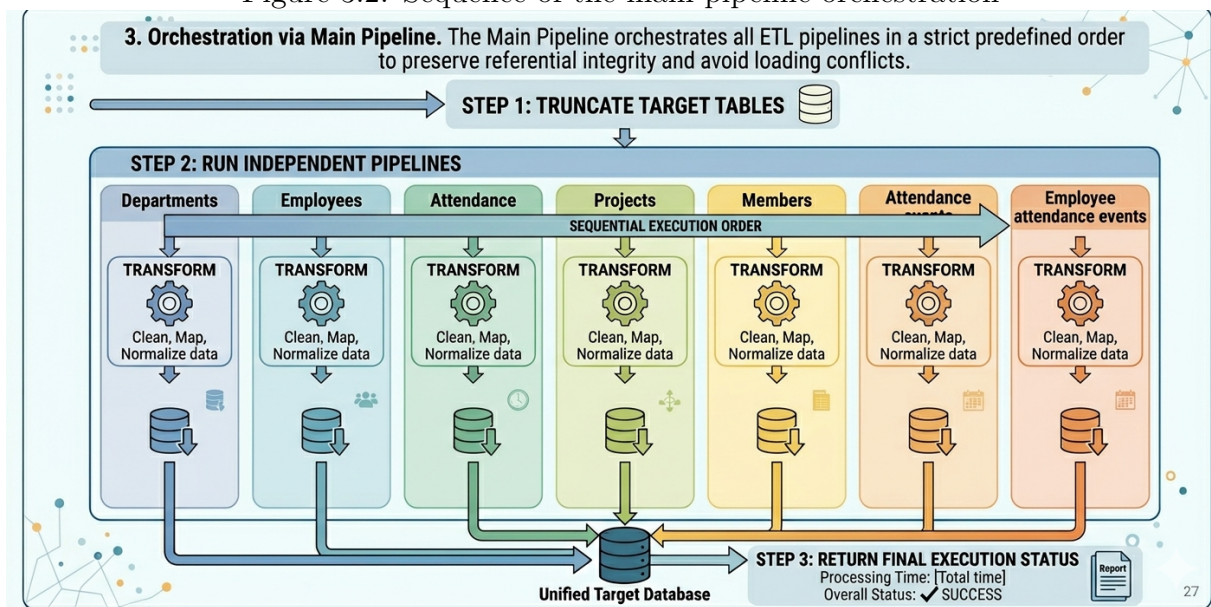
The Main Pipeline orchestrates all ETL pipelines in a strict predefined order. This order must stay unchanged to preserve referential integrity and avoid loading conflicts.

Execution flow:

1. Truncate target tables using `TRUNCATE ... CASCADE` to start from a clean load state.
2. Run pipelines in sequence: Departments → Employees → Attendance → Projects → Members → Attendance Events → Employee Attendance Events.
3. Return final execution status and total processing time.

Figure 3.2 details truncation, ordered pipeline execution, and final completion reporting.

Figure 3.2: Sequence of the main pipeline orchestration



4. Asynchronous Streaming with FastAPI

The Main Pipeline is implemented with an asynchronous generator in FastAPI to stream progress in real time.

Key features:

- A status update is emitted for each ETL step.
- Execution time is measured per step.
- Errors are captured and returned immediately.

Example streamed payload:

```
{
  "step": "employees",
  "status": "finished",
  "time": 1.23
}
```

5. Real-Time Monitoring Interface

The ETL workflow is integrated through the FastAPI backend and exposed in the Admin settings page on the frontend. This lets administrators monitor execution in real time, detect slow or failing steps, and keep full visibility over the integration process.

3.4.2 Data Validation and Control Flow

To guarantee data quality and consistency, we implemented several validation and control mechanisms:

1. Data Validation

- Verify required fields before insertion.
- Normalize data types.
- Handle missing or inconsistent values.

2. Referential Integrity

Dependencies are preserved through strict pipeline order (for example, employee records depend on department records).

3. Error Handling

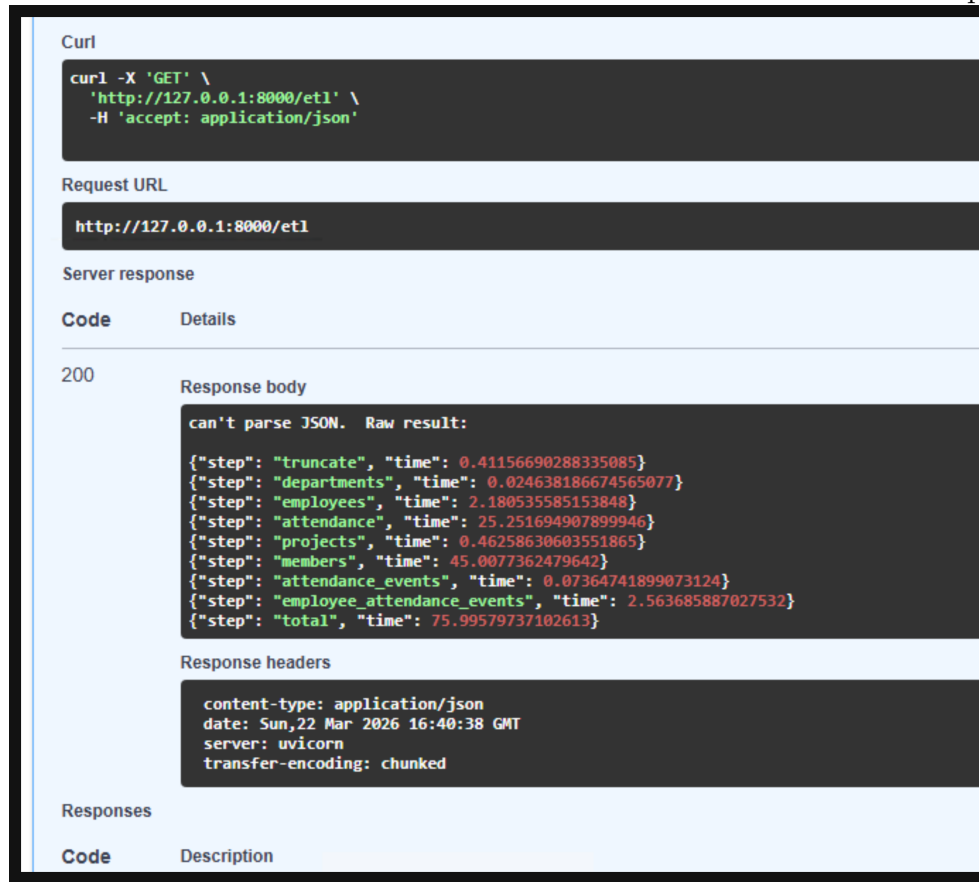
A protected try/catch orchestration flow captures failures, streams error details to the client, and raises HTTP exceptions when needed.

4. Performance Monitoring

Execution time is measured with `time.perf_counter()` for each step to identify bottlenecks and support optimization.

Figure 3.3 summarizes validation checkpoints, dependency enforcement, error propagation, and performance tracking.

Figure 3.3: Data validation and control-flow mechanisms used in the ETL process



3.4.3 Key Design Advantages

- **Modularity:** easier maintenance and extension.
- **Scalability:** new pipelines can be added with low coupling.
- **Observability:** ETL progress is streamed in real time.
- **Data Integrity:** controlled execution order prevents conflicts.
- **Performance Awareness:** per-step timing supports optimization.

3.5 Sprint Realization

3.5.1 Implementation outcomes

- Implemented source extractors and entity-based ETL pipelines.
- Implemented a main orchestrator with ordered execution and full-table refresh strategy.
- Integrated asynchronous streaming logs into backend endpoints and the admin monitoring UI.
- Applied validation, error handling, and execution-time tracking across ETL steps.

3.6 Conclusion

Sprint 3 delivered a robust and observable ETL foundation and produced the unified PostgreSQL database used by later modules. The resulting data layer ensures consistency, traceability, and operational readiness for authentication, HR, and project-management features.

Chapter 4

Authentication Module

Contents

4.1	Introduction	38
4.2	Sprint Backlog	38
4.3	Sprint Design	38
4.4	Conception	39
4.4.1	Use Case Refinement	39
4.4.2	Detailed Sequence Diagrams	41
4.5	Sprint Realization	44
4.5.1	Authentication Interface	44
4.5.2	Admin OTP Verification Interface	45
4.5.3	Registration Interface	45
4.5.4	Administrator Account Approval Interface	46
4.5.5	Approval Email Notification	46
4.5.6	Error Handling Scenarios	47
4.6	Security Considerations	47
4.7	Conclusion	48

4.1 Introduction

The authentication module controls secure access and verifies user identity. It manages registration, login, administrator approval, and an extra OTP email check for Admin login.

This module is designed to:

- Protect sensitive data.
- Restrict access to authorized users only.
- Enforce role-based access (Admin, HR, Project Manager).

The authentication flow adds one extra validation step: newly registered users must be approved by an administrator before they can access the system.

4.2 Sprint Backlog

During this sprint, we selected and implemented the following user stories:

ID	User Story	Priority
US1	As a user, I want to log in using my email and password.	High
US2	As a HR/Project Manager, I want to create an account.	High
US3	As an admin, I want to approve user accounts.	High
US4	As an admin, I want to receive an OTP code by email during login for extra verification.	High
US5	As a user, I want to receive an email after approval.	Medium
US6	As a system, I must prevent unapproved users from accessing the application.	High

4.3 Sprint Design

Login

- The user enters email and password.
- The system verifies credentials.
- If credentials are incorrect, an error message is displayed.
- If credentials are correct but the account is not approved, access is denied.
- If the user is an Admin, the system sends an OTP code to the admin email.
- The Admin enters the OTP code for an extra verification step.
- If the OTP code is correct, access is granted.
- For other approved users, access is granted directly.

Registration

- The HR or Project Manager fills the registration form.

- The system creates the account with status **Pending**.
- The user cannot log in until administrator approval.

Administrator Approval

- The administrator accesses the account approval interface.
- Pending accounts are reviewed and approved or rejected.
- On approval, the system automatically sends an email notification.

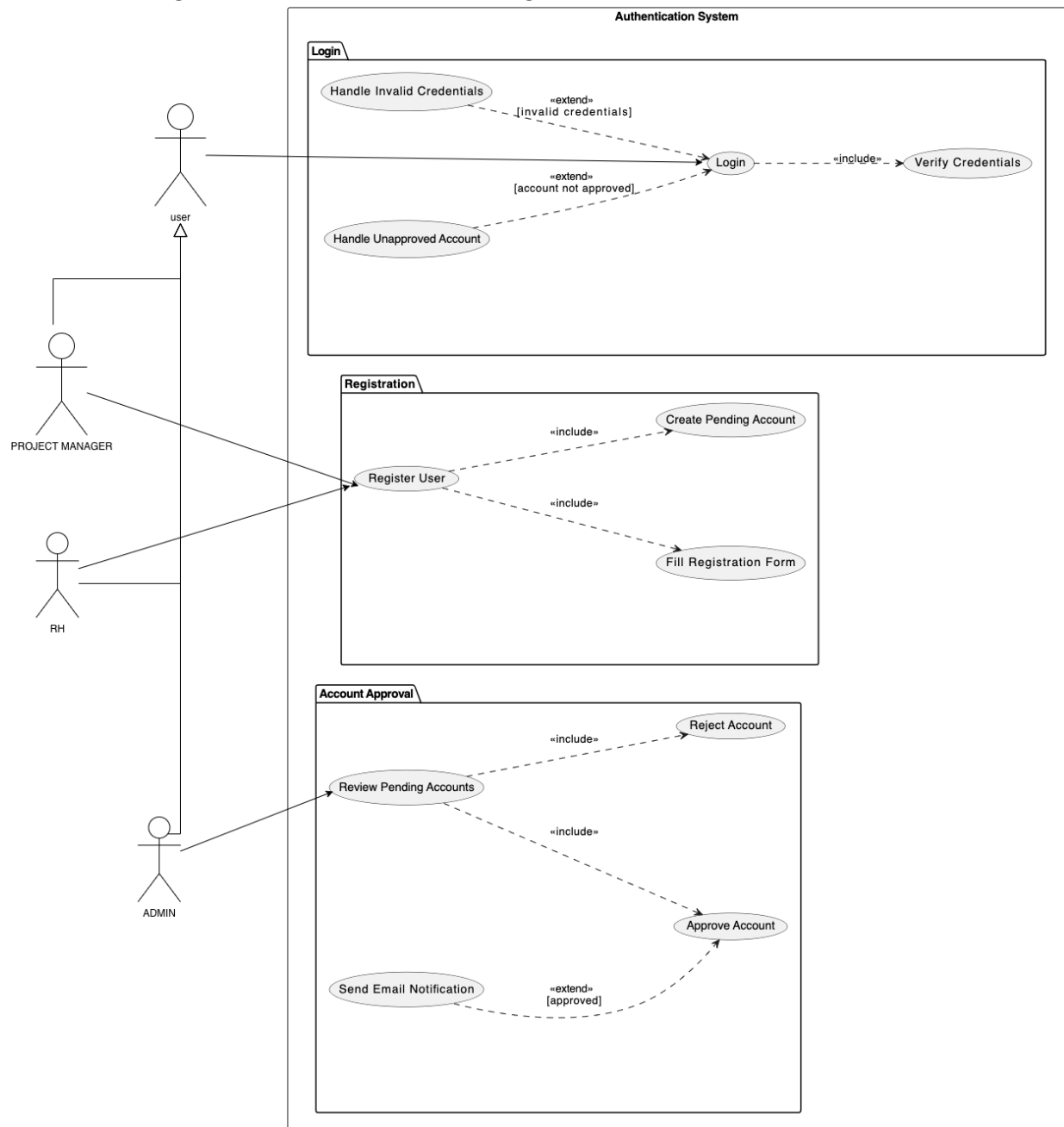
4.4 Conception

4.4.1 Use Case Refinement

The authentication system is built around the following use cases:

Figure 4.1 presents the refined authentication use cases and actor interactions used for sprint design decisions.

Figure 4.1: Refined use case diagram for the authentication module



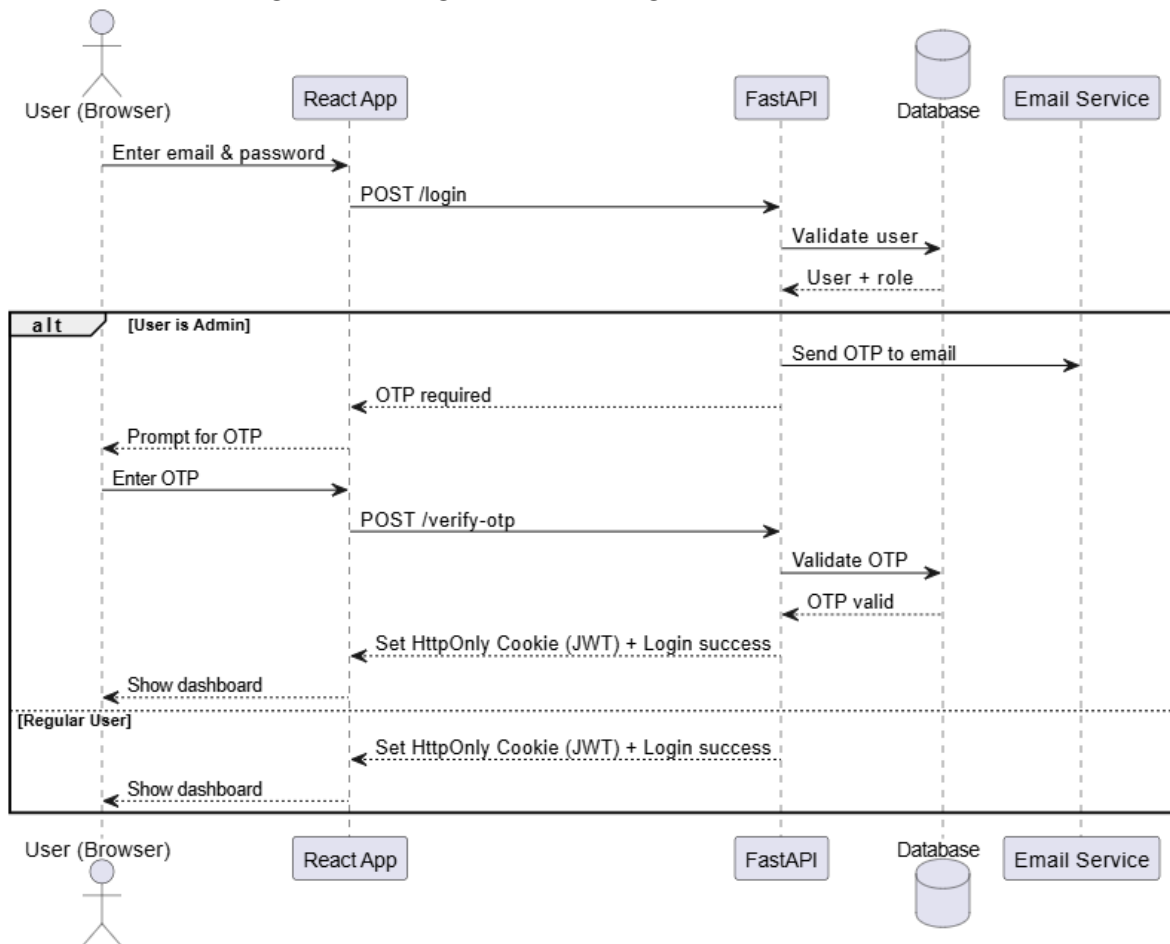
4.4.2 Detailed Sequence Diagrams

For readability, each sequence diagram is presented below with its own caption and interpretation. **Authentication flow sequence:**

1. Login and token issuance (Figure 4.2).
2. Access to protected routes (Figure 4.3).
3. Token expiration and re-authentication (Figure 4.4).
4. Logout and session invalidation (Figure 4.5).

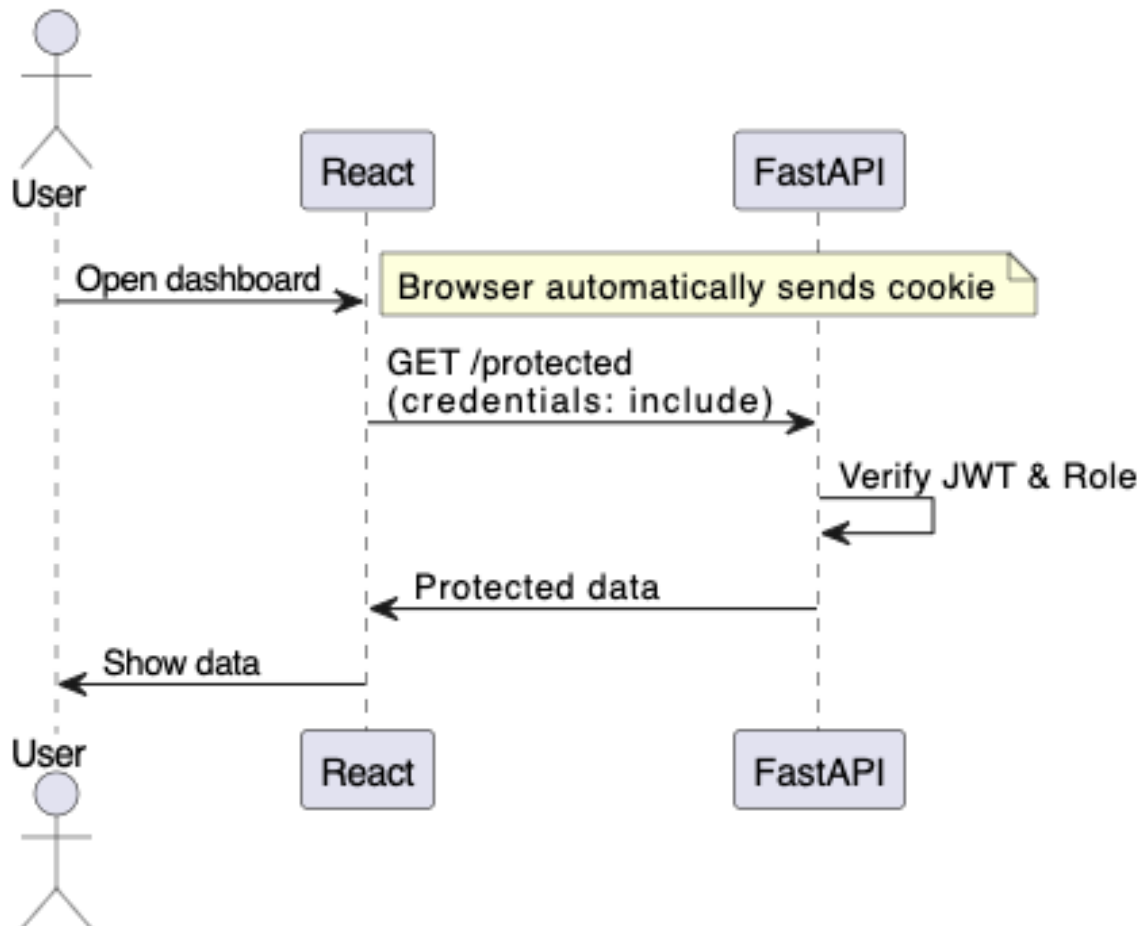
Login Sequence Diagram Figure 4.2 shows credential submission, validation, and token generation. The user sends email and password to the authentication endpoint, where credentials and account status are checked before access is granted. If the user is an Admin, the system sends an OTP code to the admin email and waits for code validation before issuing access. When validation succeeds, the backend returns a JWT used for later API calls.

Figure 4.2: Login flow showing JWT token issuance



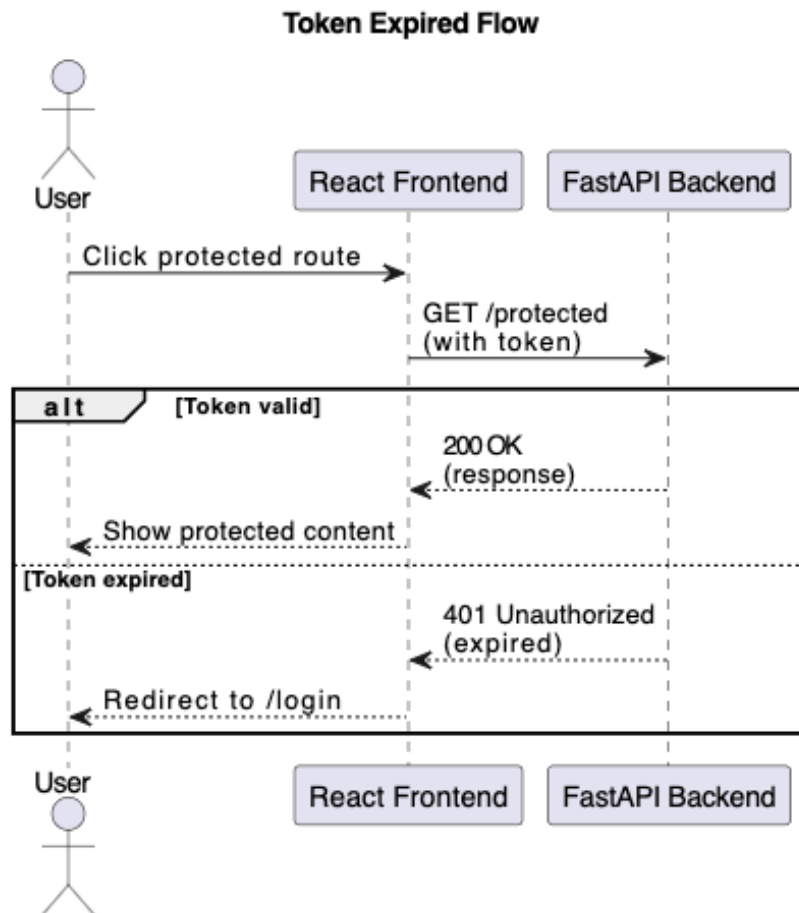
Protected Route Access Sequence Diagram Figure 4.3 explains token and role verification before protected resources are served. For each protected request, middleware validates token signature and expiration, then extracts user identity and role claims. The endpoint runs only if RBAC permissions allow access to the requested resource.

Figure 4.3: Protected route access with JWT verification



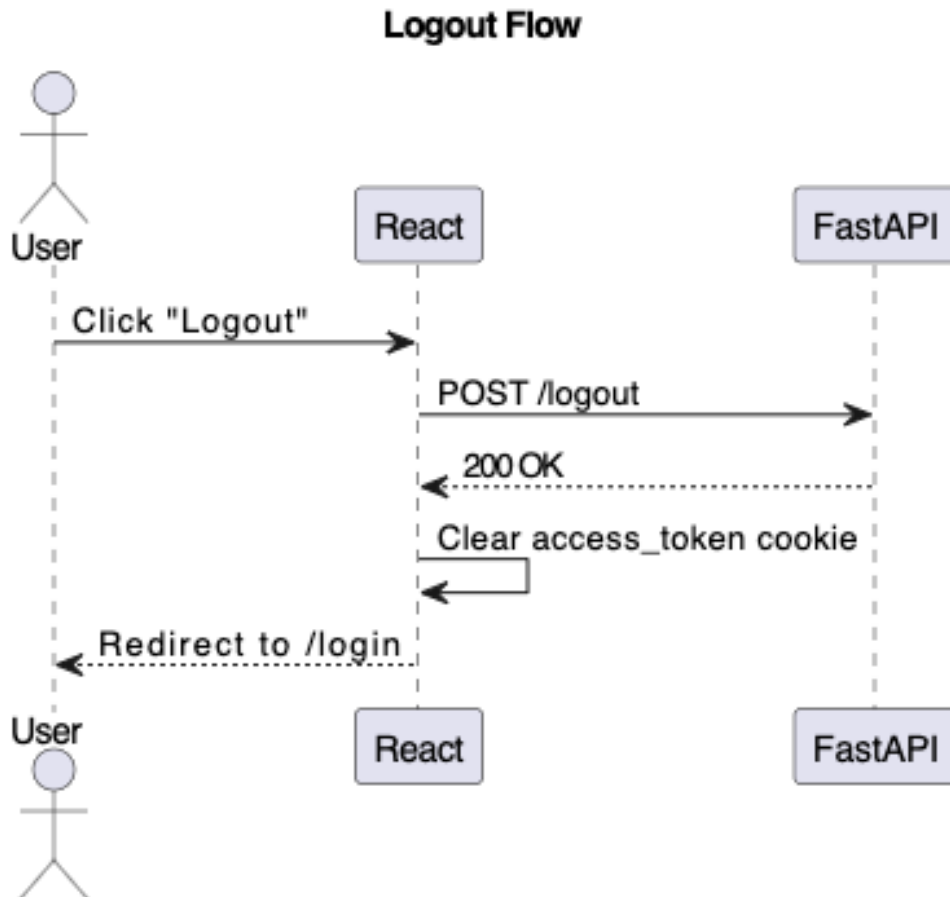
Token Expiration Sequence Diagram Figure 4.4 describes session expiration and re-authentication behavior. When the token lifetime ends, the API rejects the request with an unauthorized response and the client redirects the user to sign in again. This mechanism enforces short sessions and reduces risks linked to stale credentials.

Figure 4.4: Token expiration and re-authentication flow



Logout Sequence Diagram Figure 4.5 presents controlled session termination and token invalidation. During logout, the client clears local authentication data and stops sending the previous token in future requests. Any later access to protected routes requires a new login and a newly issued token.

Figure 4.5: Logout flow with token invalidation



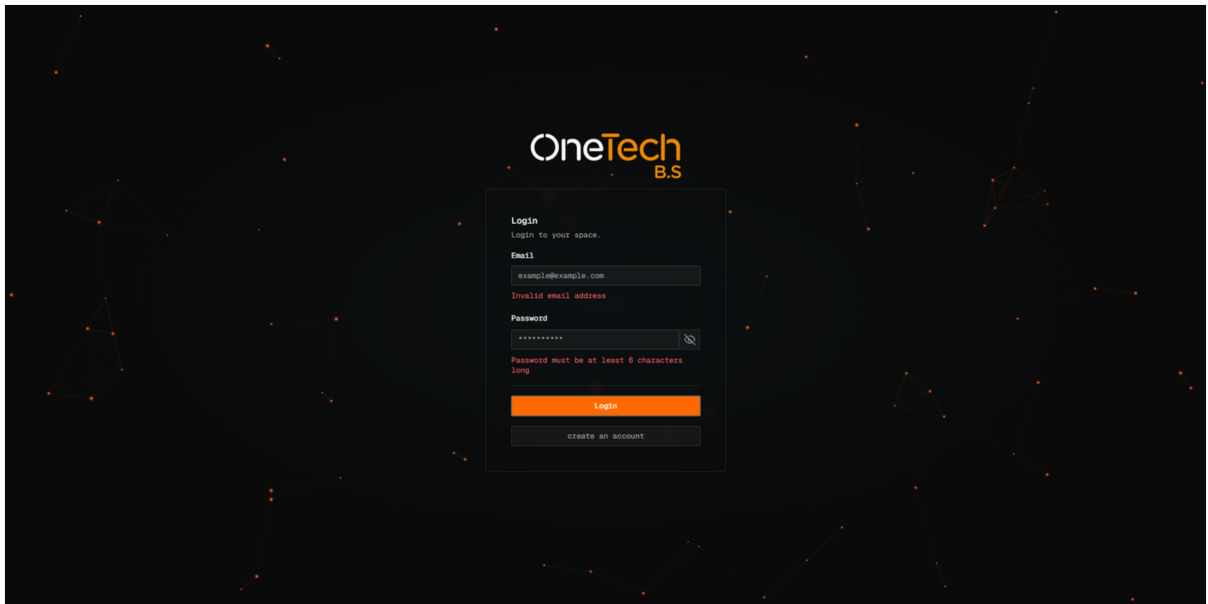
4.5 Sprint Realization

4.5.1 Authentication Interface

The implemented authentication interface includes the login view, credential validation feedback, and the OTP verification step for Admin users.

Figure 4.6 shows email/password authentication, user feedback on invalid credentials, and the Admin OTP verification step.

Figure 4.6: Login interface of the application

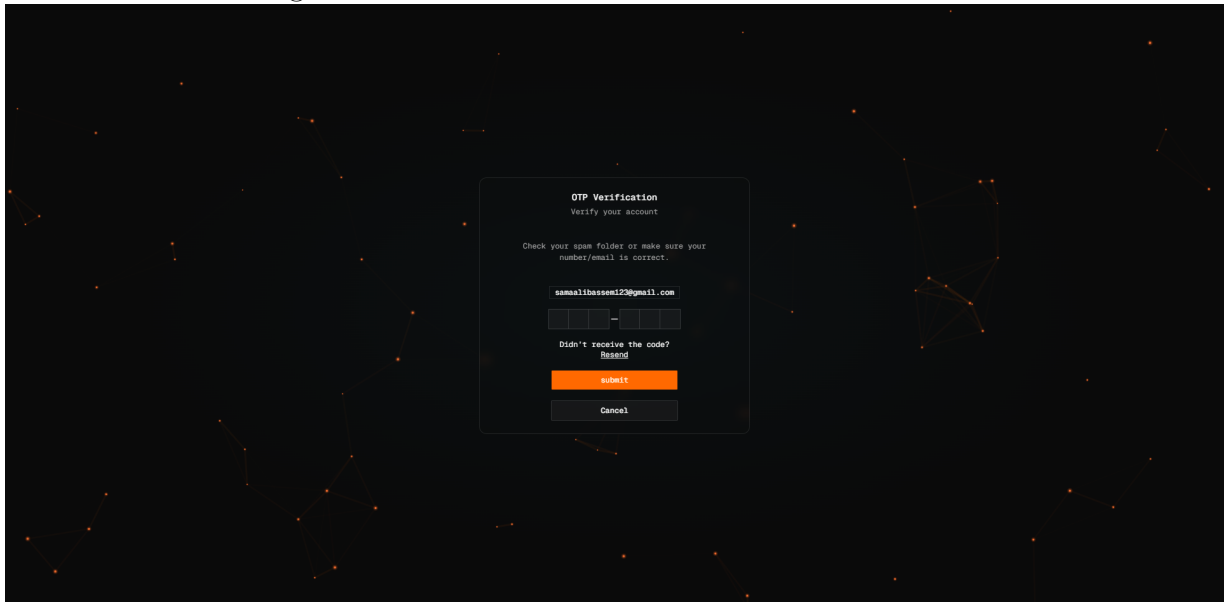


4.5.2 Admin OTP Verification Interface

After entering valid login credentials, the Admin is asked to enter the OTP code sent to their email.

Figure 4.7 presents the extra verification step where the Admin enters the code received by email before access is granted.

Figure 4.7: Administrator OTP verification interface



4.5.3 Registration Interface

The registration interface allows account creation for operational roles before administrator validation.

Figure 4.8 presents account creation for HR and Project Manager roles with pending approval status.

Figure 4.8: User registration interface

The image shows a user registration form for 'OneTech B.S.' on a dark background with a constellation pattern. At the top, a green message box states: 'Registration successful! Your account is waiting for admin approval.' The form is titled 'Register' and includes the subtext 'Create your account.' It contains the following fields: 'Username' (filled with 'test'), 'Email' (filled with 'switchfor000@gmail.com'), 'Role' (a dropdown menu set to 'Project Manager'), 'Password' (masked with dots), and 'Confirm Password' (also masked with dots). There are toggle icons for password visibility. Below the fields is an orange 'Register' button and a link that says 'already have an account? login'.

4.5.4 Administrator Account Approval Interface

After registration, accounts remain pending until administrator approval. Figure 4.9 shows the pending-user list and approval/rejection actions.

Figure 4.9: Administrator page for account approval

The image shows an administrator dashboard for 'TBS' with a sidebar containing 'Navigation', 'Dashboard', 'Pending Users', 'Projects', 'Profile', and 'Settings'. The main content area is titled 'Pendings ADMIN' and features a 'Filter By Role' dropdown. Below this is a table of pending users with columns for 'Created_at', 'Username', 'Email', 'Role', 'Status', and 'Actions'. The table contains six rows of data. The first row is highlighted in orange and shows a 'Pending' status with 'Approve' and 'Decline' buttons. The other rows show 'Declined' or 'Approved' statuses. At the bottom right of the table, there is a 'rows' dropdown and a 'page 0 of 1' indicator.

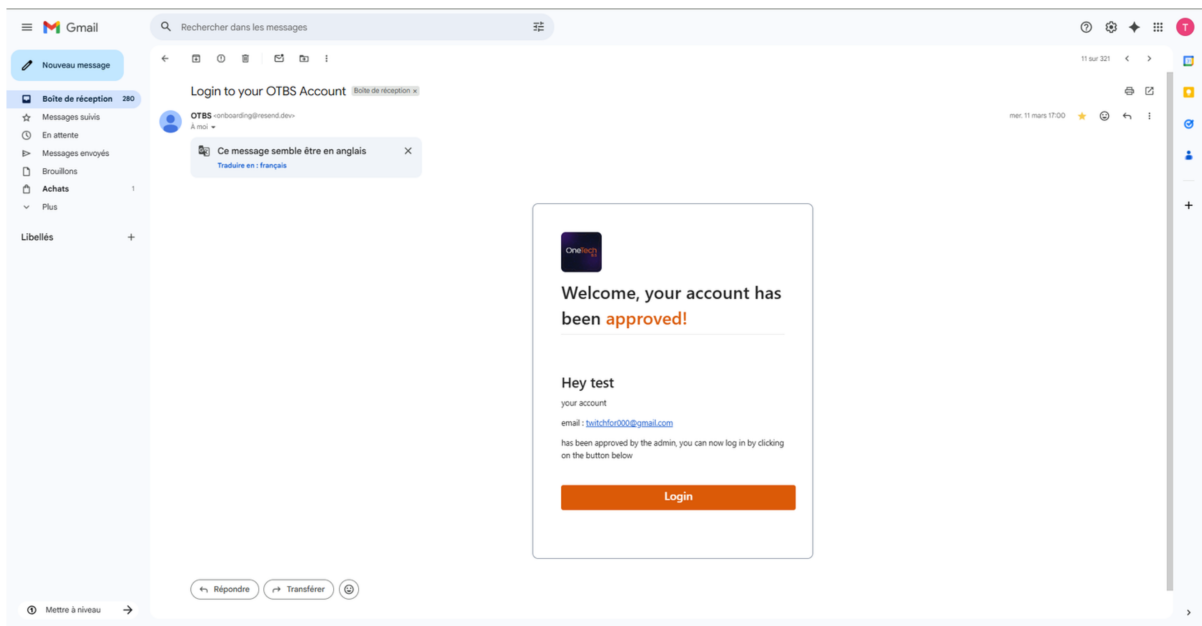
Created_at	Username	Email	Role	Status	Actions
2020-04-05 23:04:47	test	switchfor000@gmail.com	Project Manager	Pending	Approve ✓ Decline ✕
2020-03-18 22:03:19	test	rh3test@gmail.com	RH	Declined	—
2020-04-05 23:04:01	test	switchfor000@gmail.com	Project Manager	Declined	—
2020-03-07 19:03:00	test	rh3test@gmail.com	RH	Approved	—
2020-03-18 00:03:29	test	prtest@gmail.com	Project Manager	Approved	—
2020-03-11 16:03:14	test	smaailbassem123@gmail.com	RH	Approved	—

4.5.5 Approval Email Notification

When an account is approved, the system sends an automatic activation email.

Figure 4.10 confirms automated communication of account activation.

Figure 4.10: Account approval email notification



4.5.6 Error Handling Scenarios

The implemented logic provides explicit user feedback in critical cases:

- Incorrect email/password: the system displays an authentication error.
- Correct credentials with pending account: access is denied with an approval-status message.
- Admin login with invalid or expired OTP: access is denied until the correct code is entered.

4.6 Security Considerations

The following security controls were applied in the authentication module:

Control	Description
Password hashing	Passwords are stored as hashes (e.g., bcrypt) rather than plaintext.
Input validation	Form inputs are validated to prevent invalid or malformed authentication requests.
Secure error handling	Authentication errors do not expose sensitive system details.
RBAC enforcement	Access to protected resources depends on verified user role permissions.
Approval-gated access	Users cannot access business modules unless account status is approved by the administrator.
Admin OTP verification	Admin users must validate a one-time password sent by email before login is completed.

4.7 Conclusion

The authentication module ensures secure and controlled access to the application. With administrator approval, automatic email notification, and OTP verification for Admin users, the system adds an extra layer of protection for sensitive access.

Chapter 5

Administration Module

Contents

5.1	Introduction	50
5.2	Sprint Backlog	50
5.3	Sprint Design	50
5.3.1	Dashboard module design	50
5.3.2	Projects module design	51
5.4	Conception	51
5.4.1	Administration use case diagram	51
5.5	Sprint Realization	52
5.5.1	Implementation outcomes	52
5.5.2	Administration dashboard and users interface	53
5.5.3	Projects interface	53
5.5.4	Assign project interface	54
5.5.5	Settings page interface	54
5.5.6	Filtering and pagination interface	55
5.6	Conclusion	55

5.1 Introduction

This chapter presents the Administration module, which gives administrators full control over user governance and project-manager assignments. The module is organized around two main pages: Dashboard and Projects.

5.2 Sprint Backlog

The administration backlog focused on governance and control features:

ID	Backlog Item	Priority
AB-01	Build a dashboard to monitor account distribution by role (Admin, HR, Project Manager).	High
AB-02	Allow administrators to create, edit, and delete user accounts.	High
AB-03	Provide controlled user editing while preventing direct password modification from the listing workflow.	Medium
AB-04	Allow administrators to assign project managers to projects.	High
AB-05	Add filtering, pagination, and scalable table behavior for large datasets.	Medium

5.3 Sprint Design

The module design separates account governance and project assignment into dedicated interfaces for clarity and operational efficiency. To communicate the React implementation clearly, this section presents interface mockups created with Excalidraw.

5.3.1 Dashboard module design

The Dashboard page was designed as the administration control center. It contains user summary cards (Admin, HR, and Project Manager counts) and a users table with account-management actions.

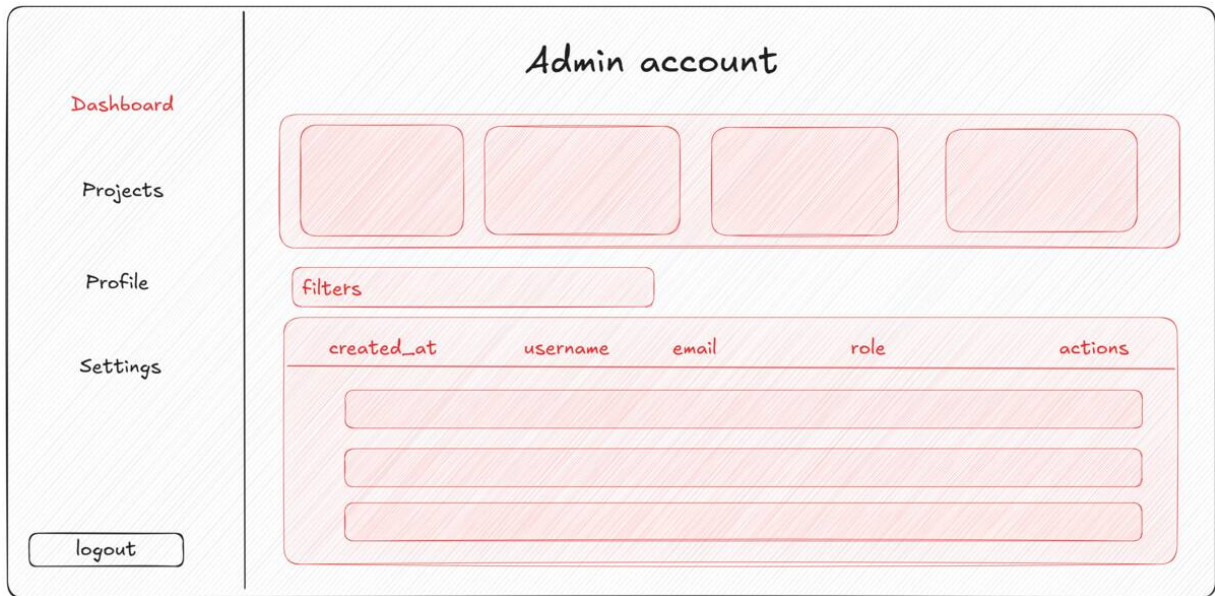


Figure 5.1: Dashboard Module Design Mockup

5.3.2 Projects module design

The Projects page was designed to manage project ownership by project managers. It contains a projects table where each row provides an action that opens a dedicated assignment page.

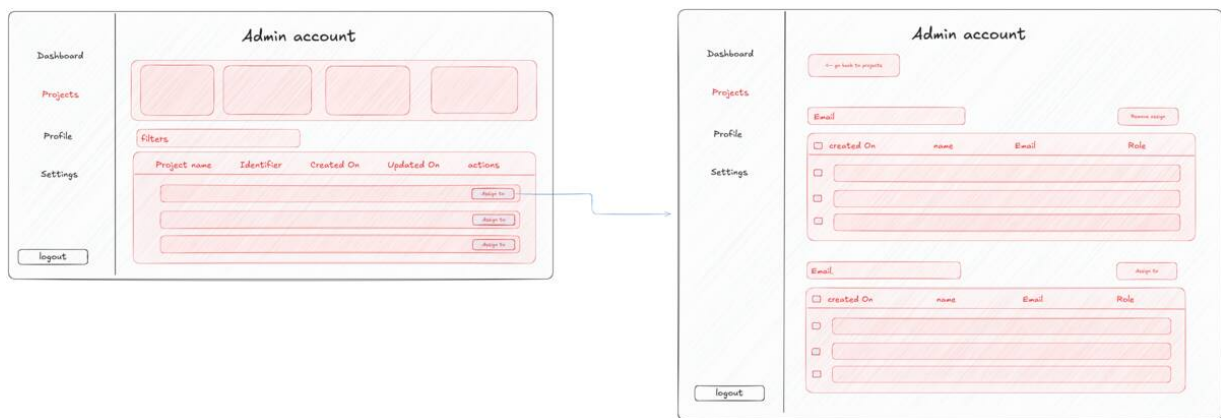


Figure 5.2: Projects Module Design Mockup

5.4 Conception

5.4.1 Administration use case diagram

The administration use case model defines the main administrator interactions: create, edit, and delete accounts; view role distribution; consult projects; and assign project managers.

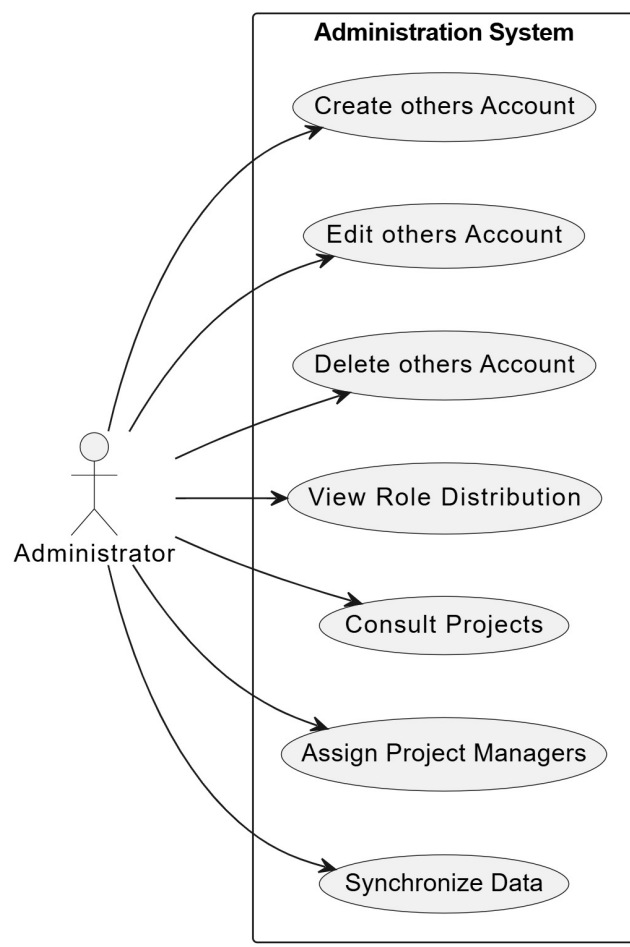


Figure 5.3: Administration Module Use Case Diagram

Figure 5.3 summarizes administrator actions and governance scope.

5.5 Sprint Realization

5.5.1 Implementation outcomes

- Implemented Dashboard module with role-based user summary cards.
- Implemented Users table with create, edit, and delete account actions.
- Implemented edit flow for user information without password update through table actions.
- Implemented Projects table and project-manager assignment page.
- Implemented Settings page actions to synchronize and update unified database data from BioTime and EasyProject.
- Integrated filters, pagination, and TanStack Query caching across tables.

5.5.2 Administration dashboard and users interface

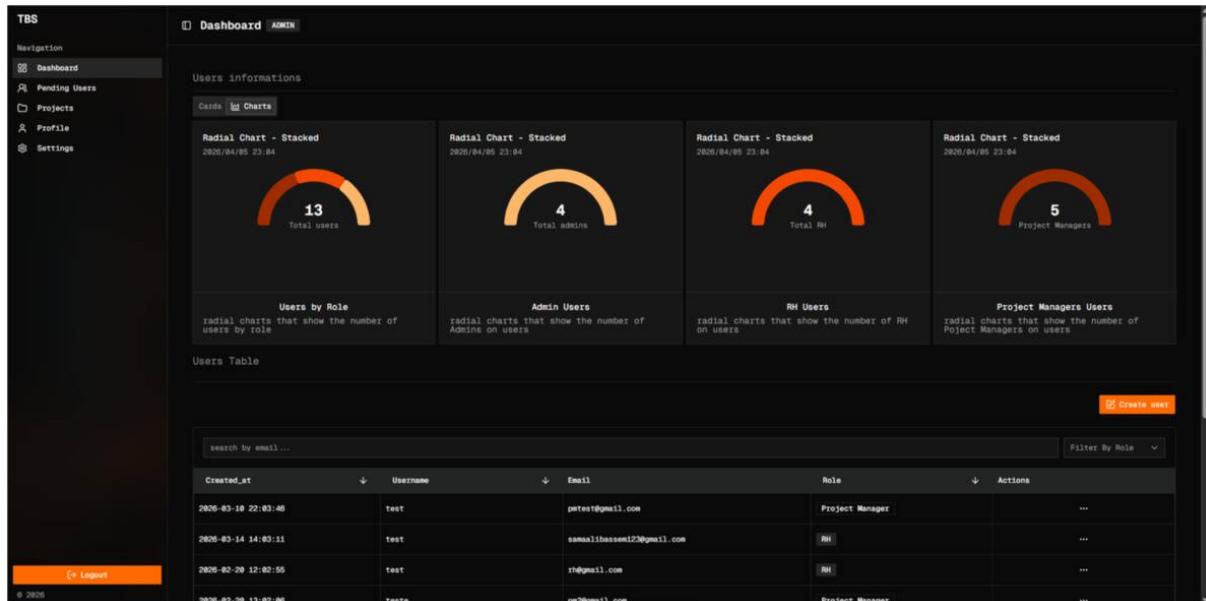


Figure 5.4: Administration Dashboard: User Cards and Users Table

Figure 5.4 shows role-distribution cards and the user-management table with edit/delete actions.

5.5.3 Projects interface

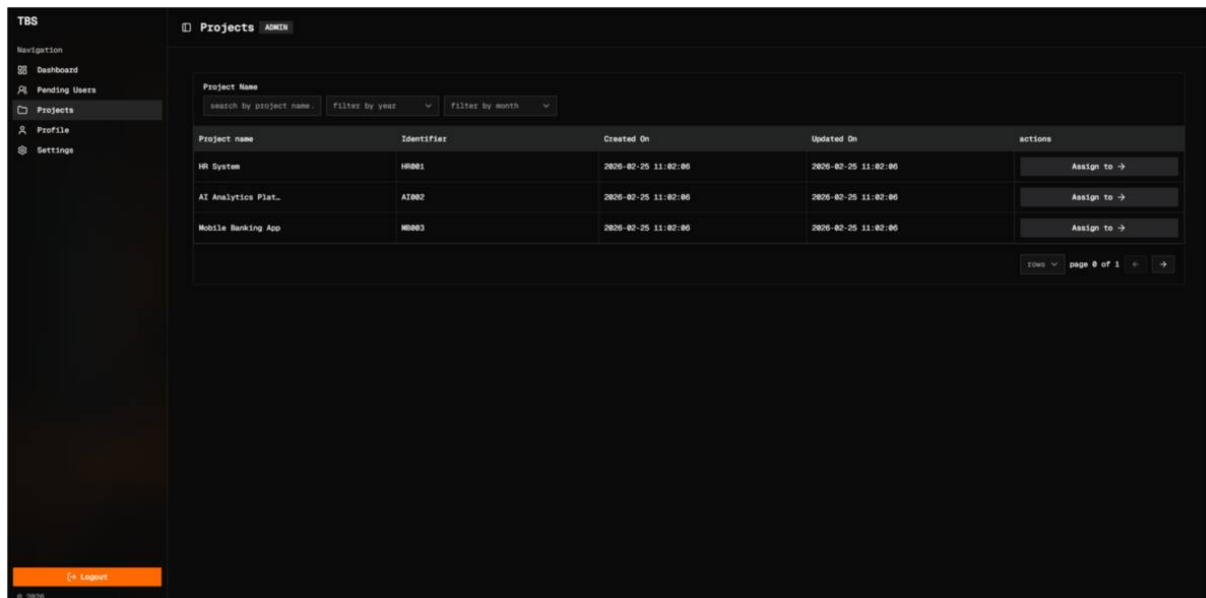


Figure 5.5: Administration Projects Page

Figure 5.5 presents the projects table used to select projects for assignment actions.

5.5.4 Assign project interface

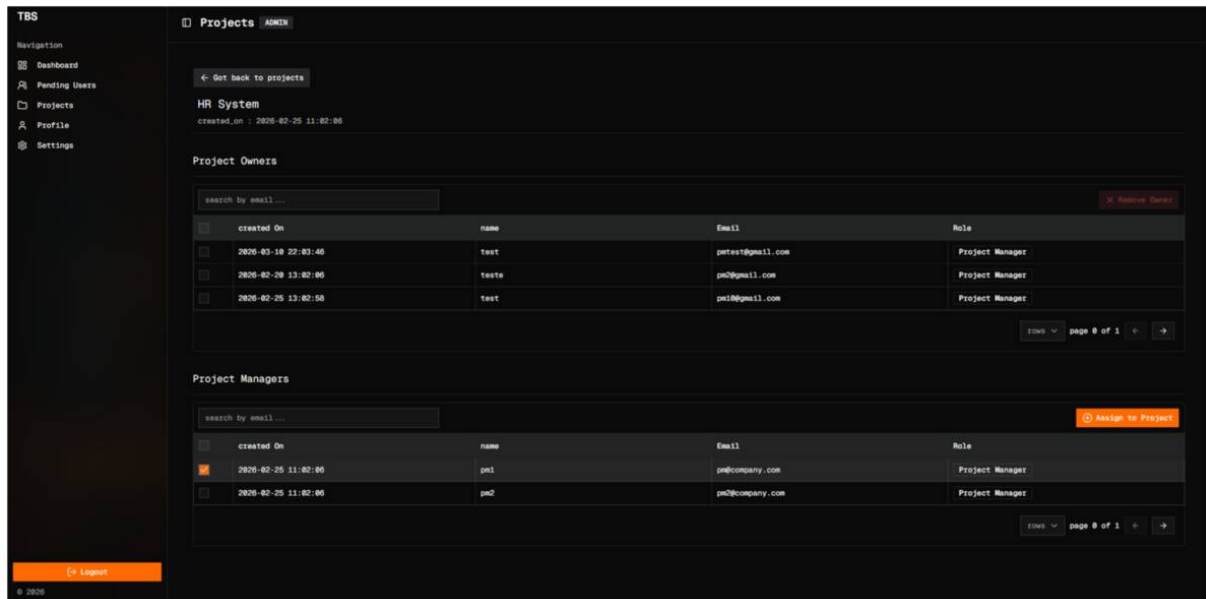


Figure 5.6: Assign Project Page

Figure 5.6 illustrates assigning one or more project managers to the selected project.

5.5.5 Settings page interface

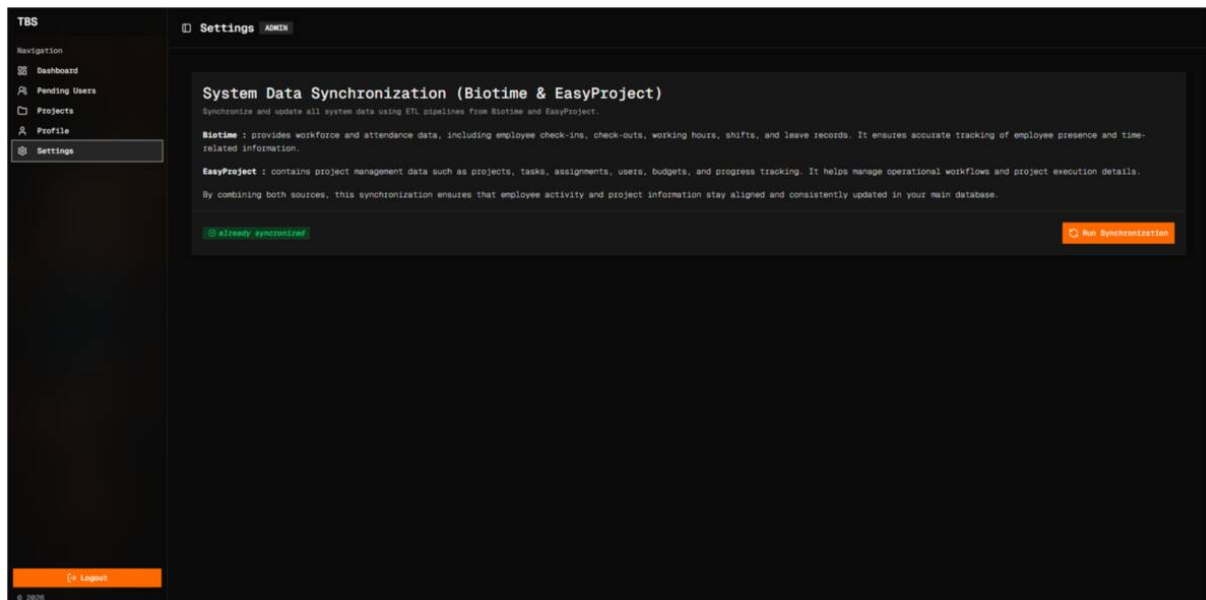


Figure 5.7: Administration Settings Page for Data Synchronization

Figure 5.7 presents the Settings interface where the administrator can trigger synchronization or update operations to refresh the unified database from BioTime and EasyProject.

5.5.6 Filtering and pagination interface

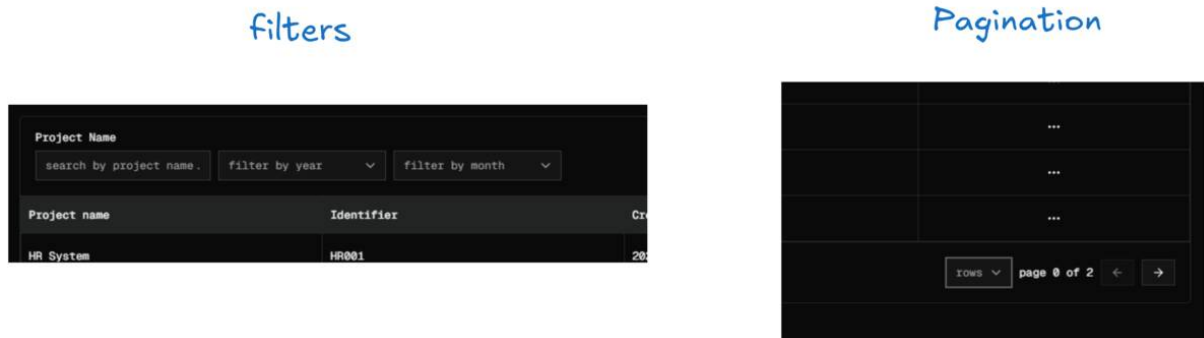


Figure 5.8: Filtering and Pagination

Figure 5.8 summarizes large-dataset handling through filters, pagination, and TanStack Query caching.

5.6 Conclusion

The Administration module delivered complete governance capabilities for users and project assignments. With scalable tables, controlled actions, and cached data access, the module supports efficient administration even with large datasets.

Chapter 6

HR Functional Modules

Contents

6.1	Introduction	57
6.2	Sprint Backlog	57
6.3	Sprint Design	57
6.3.1	Employees page design mockup	57
6.3.2	Check-In/Out page design mockup	58
6.3.3	Employee Leave page design mockup	58
6.4	Conception	59
6.4.1	HR use case diagram	59
6.5	Sprint Realization	60
6.5.1	Implementation outcomes	60
6.5.2	Employees page interface	61
6.5.3	Check-In/Out page interface	61
6.5.4	Employee Leave page interface	62
6.6	Conclusion	62

6.1 Introduction

This chapter presents the HR Functional Modules, designed to support organization-wide employee monitoring and follow-up. The module is organized into three main pages: Employees, Check-In/Out, and Employee Leave.

6.2 Sprint Backlog

The HR backlog focused on visibility, traceability, and fast navigation across workforce data.

ID	Backlog Item	Priority
HR-BL-01	Implement an Employees page with searchable and filterable employee directory data.	High
HR-BL-02	Add department-based analytics using a donut chart for employee distribution.	Medium
HR-BL-03	Implement a Check-In/Out page for attendance follow-up with date-based filtering.	High
HR-BL-04	Implement an Employee Leave page to explain missing check-in/out events.	High
HR-BL-05	Add cross-page navigation from attendance rows to pre-filtered leave results.	Medium
HR-BL-06	Ensure pagination and filtering for large datasets using scalable table technologies.	High

6.3 Sprint Design

The design objective was to give HR users a fast workflow from overview to diagnosis: identify employees, inspect attendance events, then check leave reasons when anomalies appear.

6.3.1 Employees page design mockup

The Employees page mockup combines a searchable employee directory with a department-level visualization, helping HR users get a quick organizational overview before deeper attendance analysis.

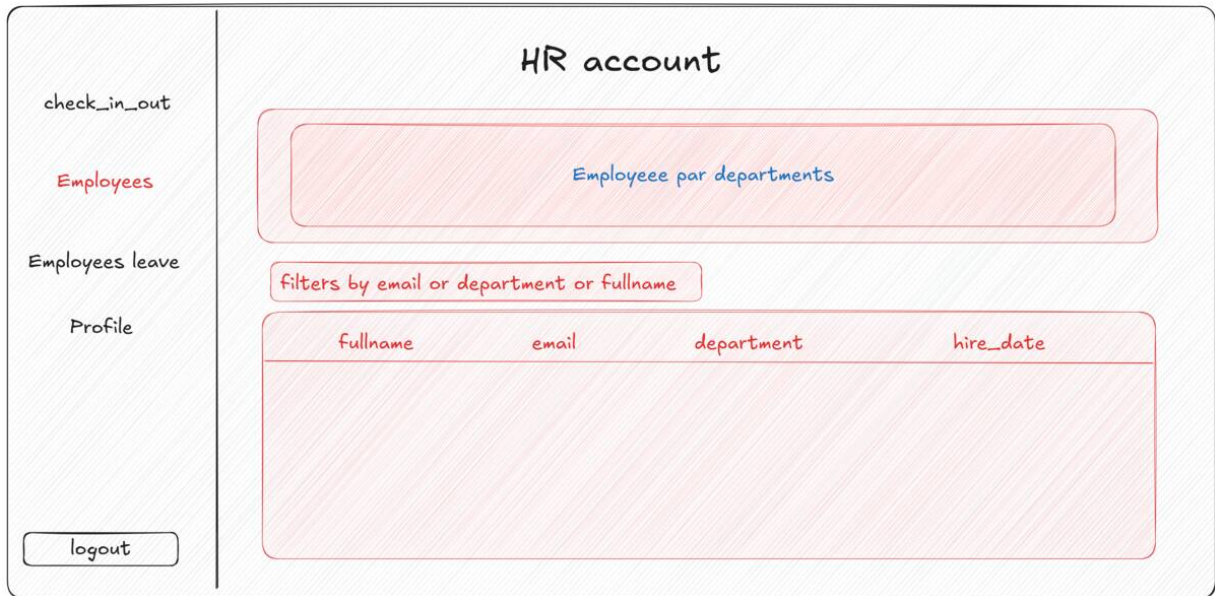


Figure 6.1: Employees Page Design Mockup

6.3.2 Check-In/Out page design mockup

The Check-In/Out page mockup focuses on daily attendance tracking, with clear check-in/check-out records and date-based filters to support operational follow-up by HR users.

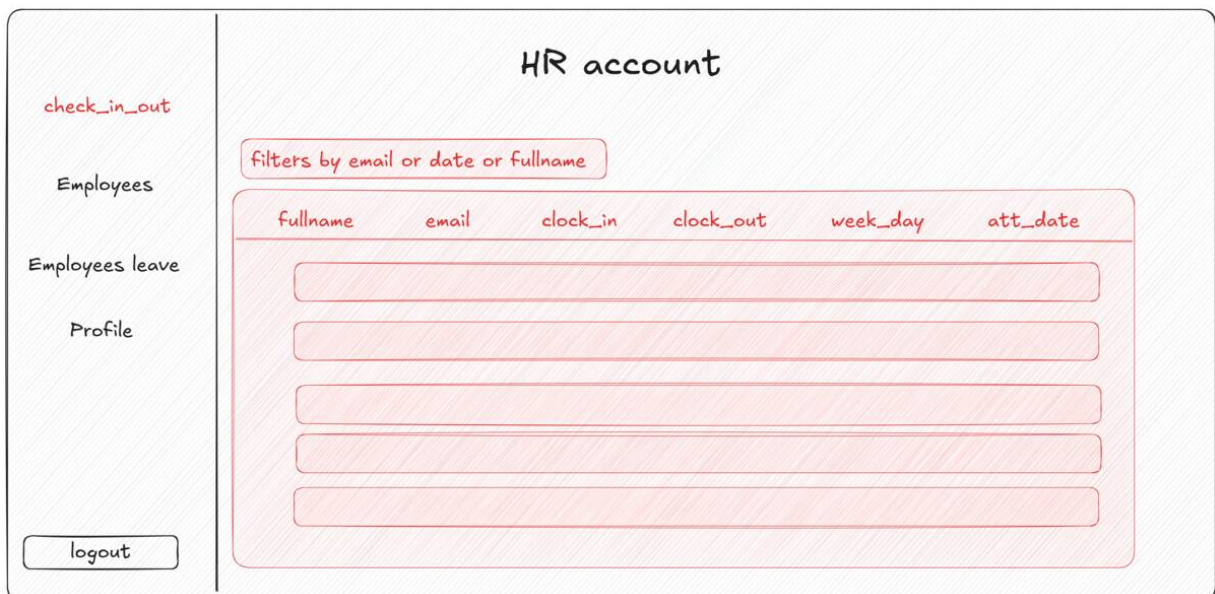


Figure 6.2: Check-In/Out Page Design Mockup

6.3.3 Employee Leave page design mockup

The Employee Leave page mockup explains attendance anomalies by showing leave periods and reasons in a readable, filterable layout linked to attendance investigations.

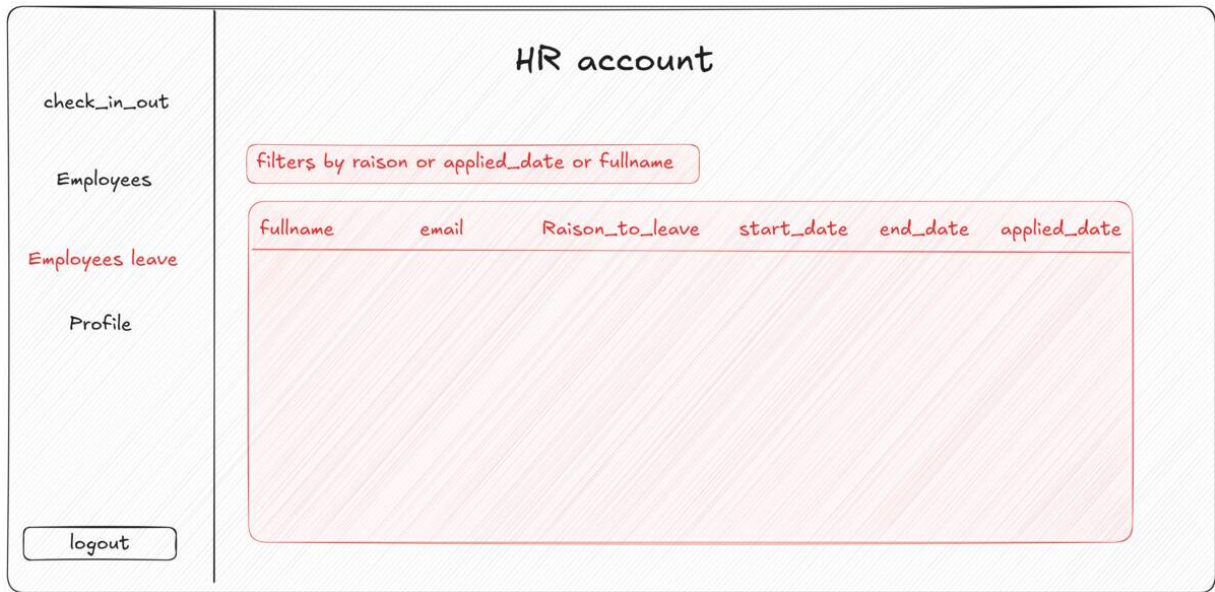


Figure 6.3: Employee Leave Page Design Mockup

6.4 Conception

6.4.1 HR use case diagram

The HR use case model defines the main HR interactions: consult employees, analyze attendance check-in/out records, inspect leave reasons, and navigate between related records.

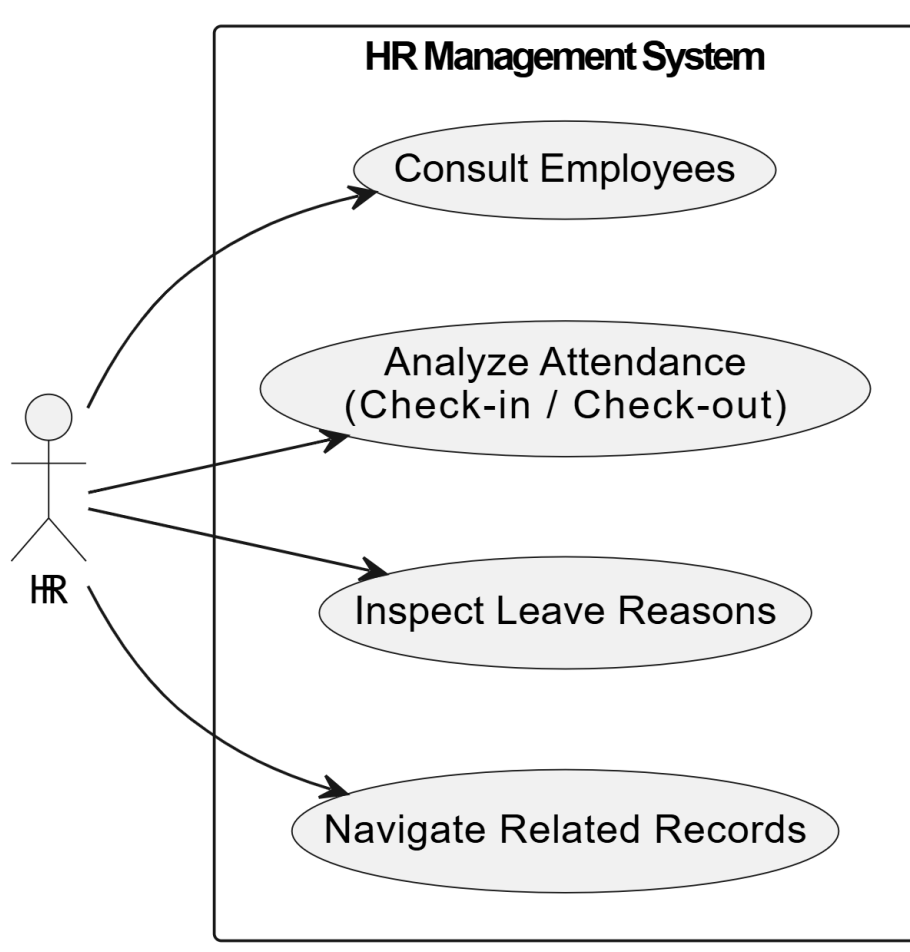


Figure 6.4: HR Functional Modules Use Case Diagram

Figure 6.4 summarizes the HR workflow across employee consultation, attendance tracking, and leave analysis.

6.5 Sprint Realization

6.5.1 Implementation outcomes

- Implemented Employees page with filters (name, email, department) and department donut chart.
- Implemented Check-In/Out page with attendance events and date-based filters.
- Implemented Employee Leave page with leave reasons linked to attendance gaps.
- Implemented redirect behavior from Check-In/Out rows to a pre-filtered Employee Leave view.
- Integrated pagination, filtering, TanStack Table, and TanStack Query caching across HR tables.

6.5.2 Employees page interface

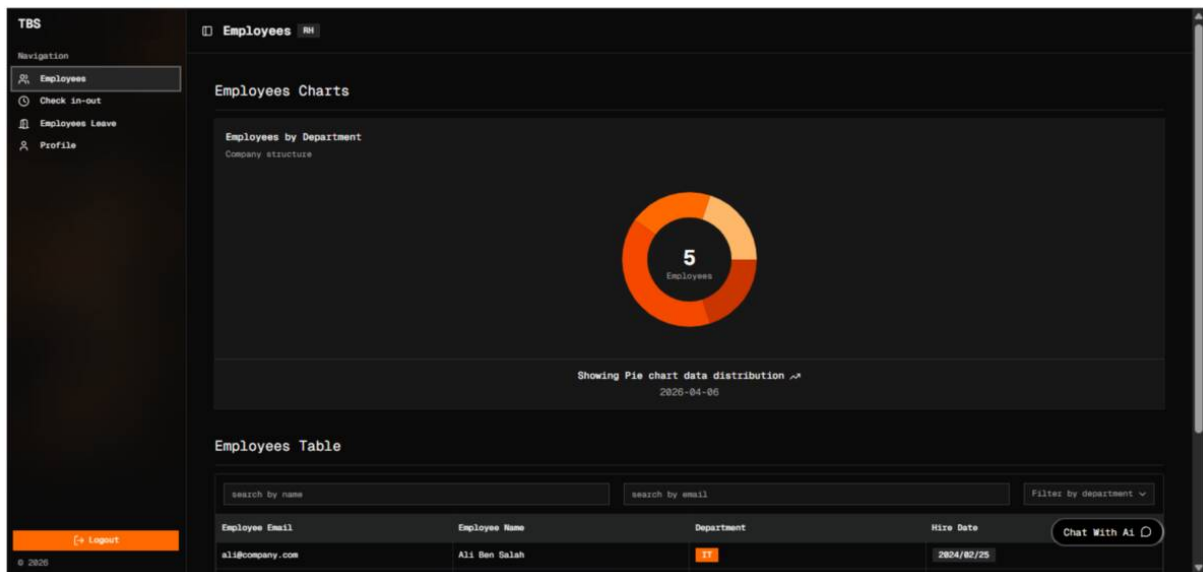


Figure 6.5: HR Employees Page: Employee Directory and Department Donut Chart

Figure 6.5 shows the employee directory with filters and the donut chart used for department-level workforce distribution.

6.5.3 Check-In/Out page interface

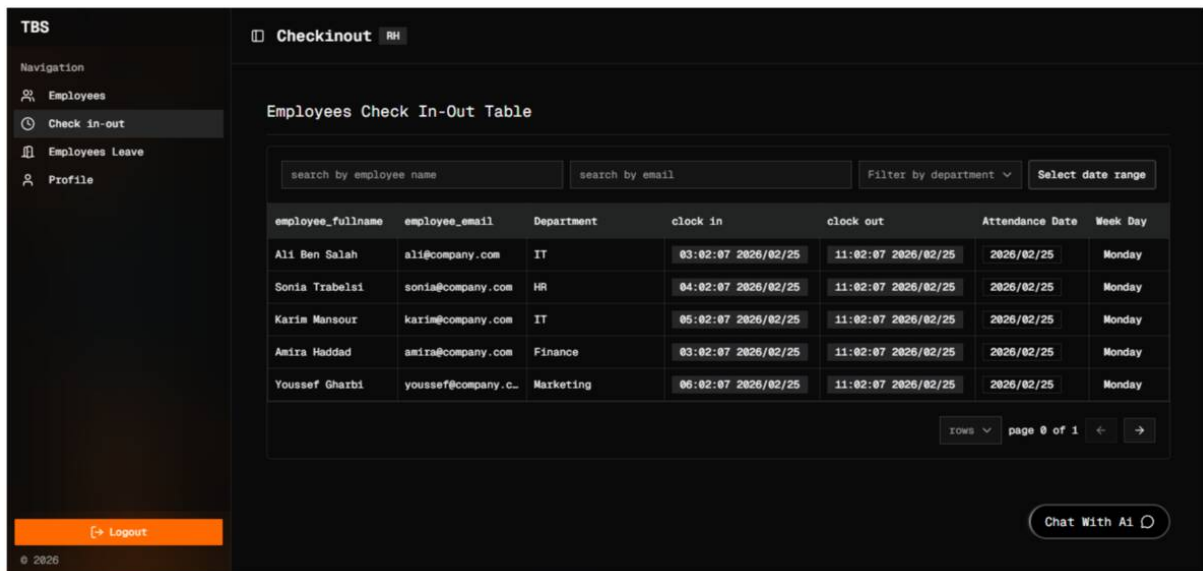


Figure 6.6: HR Check-In/Out Page

Figure 6.6 illustrates attendance events with date filters for operational monitoring.

6.5.4 Employee Leave page interface

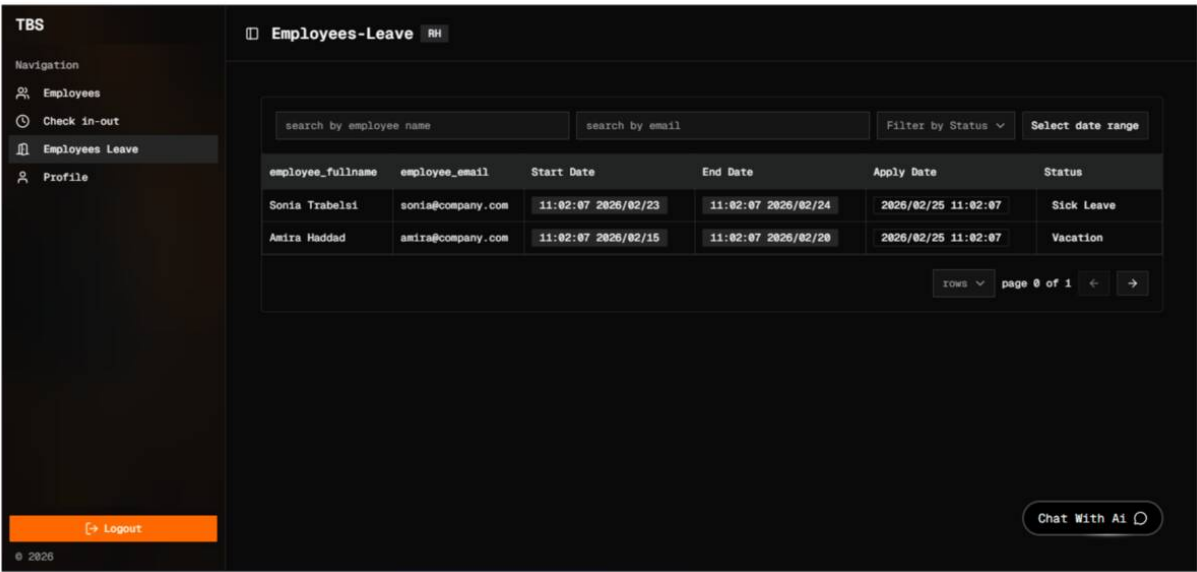


Figure 6.7: HR Employee Leave Page

Figure 6.7 presents leave details and reasons linked to missing attendance actions.

6.6 Conclusion

The HR Functional Modules delivered a complete workflow for employee monitoring, attendance follow-up, and leave-based explanation of missing events. With scalable tables, smart navigation, and cached data access, the module supports efficient HR operations on large datasets.

Chapter 7

Project Manager Modules

Contents

7.1	Introduction	64
7.2	Sprint Backlog	64
7.3	Sprint Design	64
7.3.1	Projects page design mockup	64
7.3.2	Members page design mockup	65
7.3.3	Members Attendance page design mockup	65
7.4	Conception	66
7.4.1	Project Manager use case diagram	66
7.5	Sprint Realization	67
7.5.1	Implementation outcomes	67
7.5.2	Projects page interface	68
7.5.3	Members page interface	68
7.5.4	Members attendance monitoring interface	69
7.6	Conclusion	69

7.1 Introduction

This chapter presents the Project Manager modules, focused on project-scope supervision and manager-level operational monitoring. The Project Manager has access to three pages: **Projects**, **Members**, and **Members Attendance**. Access is strictly limited to projects assigned by the Administrator and members belonging to those projects.

7.2 Sprint Backlog

The sprint backlog prioritized project-management monitoring tasks.

ID	Backlog Item	Priority
PM-BL-01	Implement the Projects page to list only Administrator-assigned projects.	High
PM-BL-02	Implement the Members page to display members of assigned projects only.	High
PM-BL-03	Implement the Members Attendance page to monitor attendance of assigned-project members.	High
PM-BL-04	Execute module stabilization and end-to-end verification.	Medium

7.3 Sprint Design

The sprint design centered on role-scoped visibility, operational clarity, and module interoperability.

7.3.1 Projects page design mockup

The Projects page mockup provides a clear, role-scoped list of projects assigned by the Administrator, allowing managers to quickly identify project context before exploring members and attendance details.

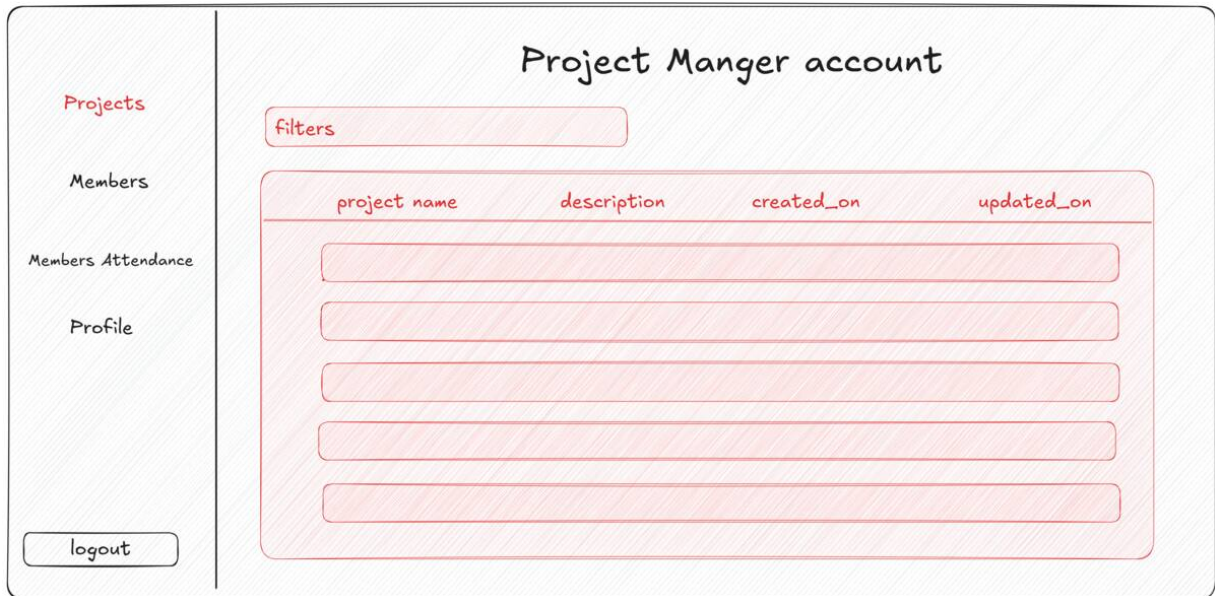


Figure 7.1: Project Manager Projects Page Design Mockup

7.3.2 Members page design mockup

The Members page mockup shows members linked to the selected assigned project, with clear tabular visibility for team composition and quick project-level supervision.

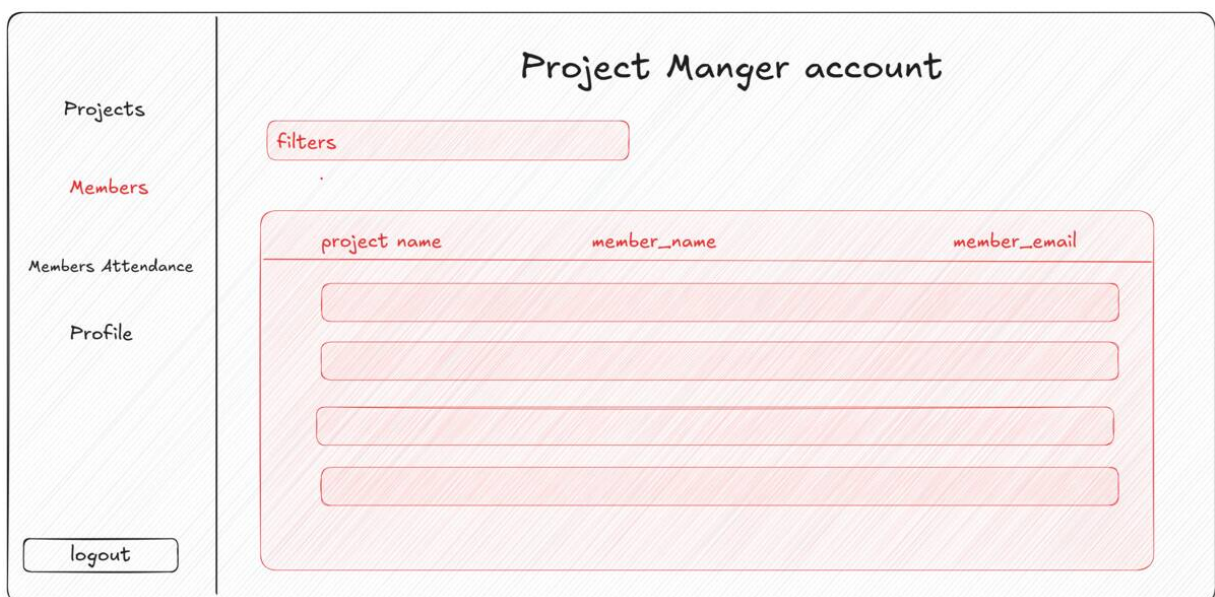


Figure 7.2: Project Manager Members Page Design Mockup

7.3.3 Members Attendance page design mockup

The Members Attendance page mockup supports attendance follow-up on members of assigned projects only, with clear records for manager-level operational tracking.

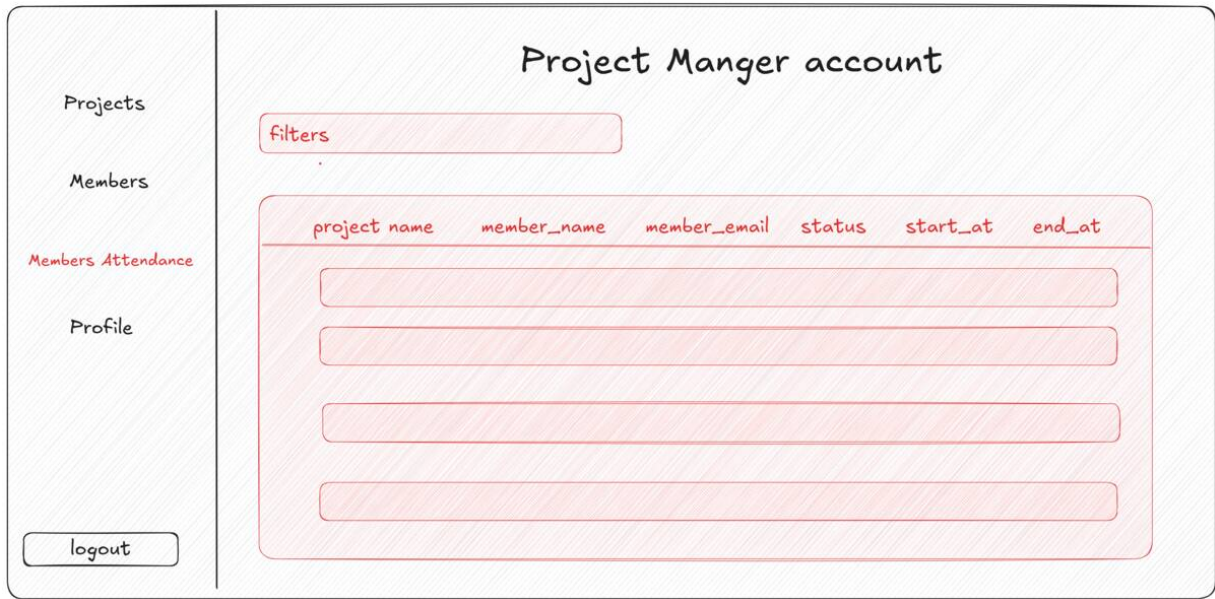


Figure 7.3: Project Manager Members Attendance Page Design Mockup

7.4 Conception

7.4.1 Project Manager use case diagram

The Project Manager use case model captures role-scoped actions: consult assigned projects, view members of those projects, and monitor attendance for those members only.

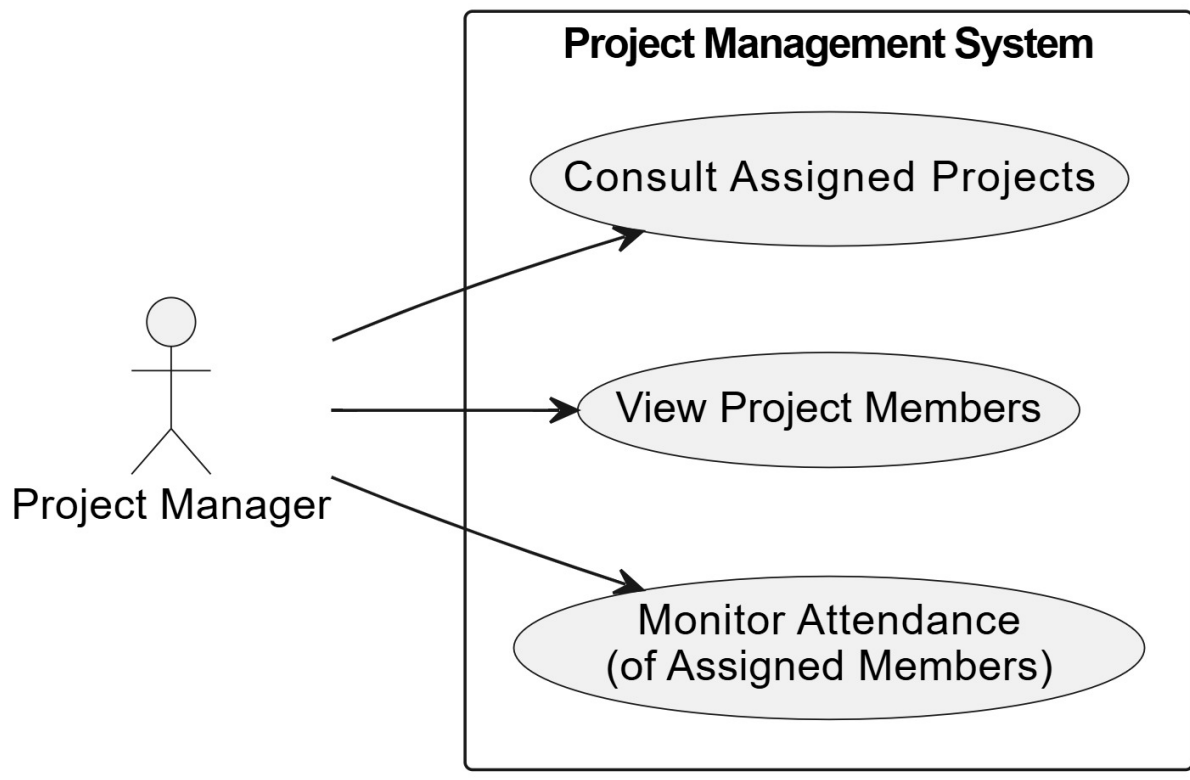


Figure 7.4: Project Manager Modules Use Case Diagram

Figure 7.4 summarizes manager-level actions constrained to assigned projects.

7.5 Sprint Realization

7.5.1 Implementation outcomes

- Implemented the Projects page with visibility restricted to Administrator-assigned projects.
- Implemented the Members page with members filtered by assigned projects.
- Implemented the Members Attendance page with attendance records limited to assigned-project members.
- Added tabular querying features for project-level supervision.

7.5.2 Projects page interface

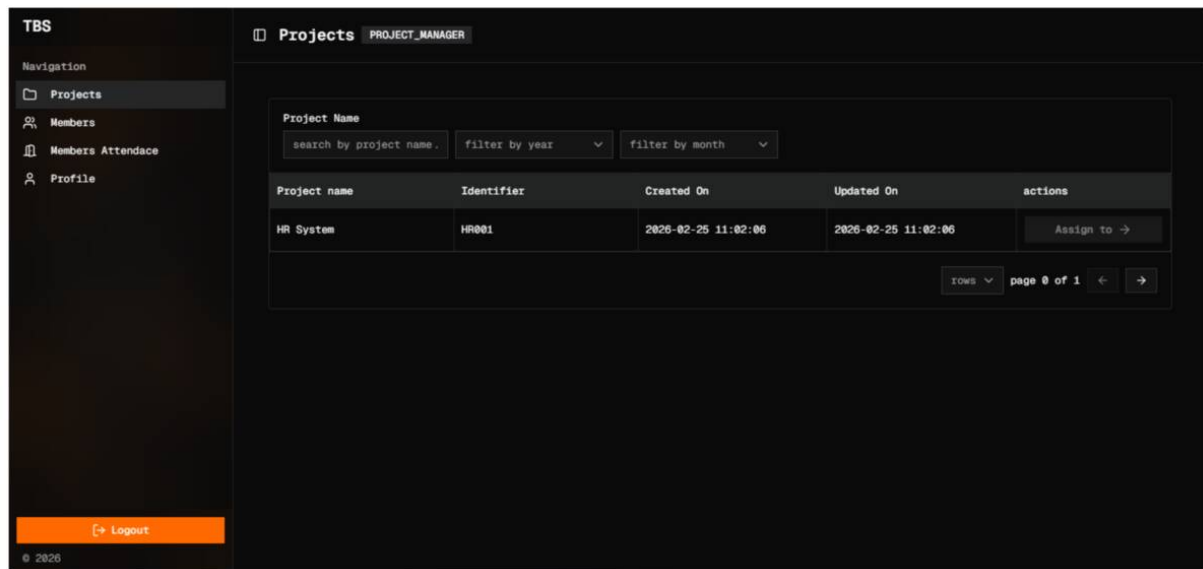


Figure 7.5: Project Manager Projects Page

Figure 7.5 shows assigned-project consultation for manager scope.

7.5.3 Members page interface

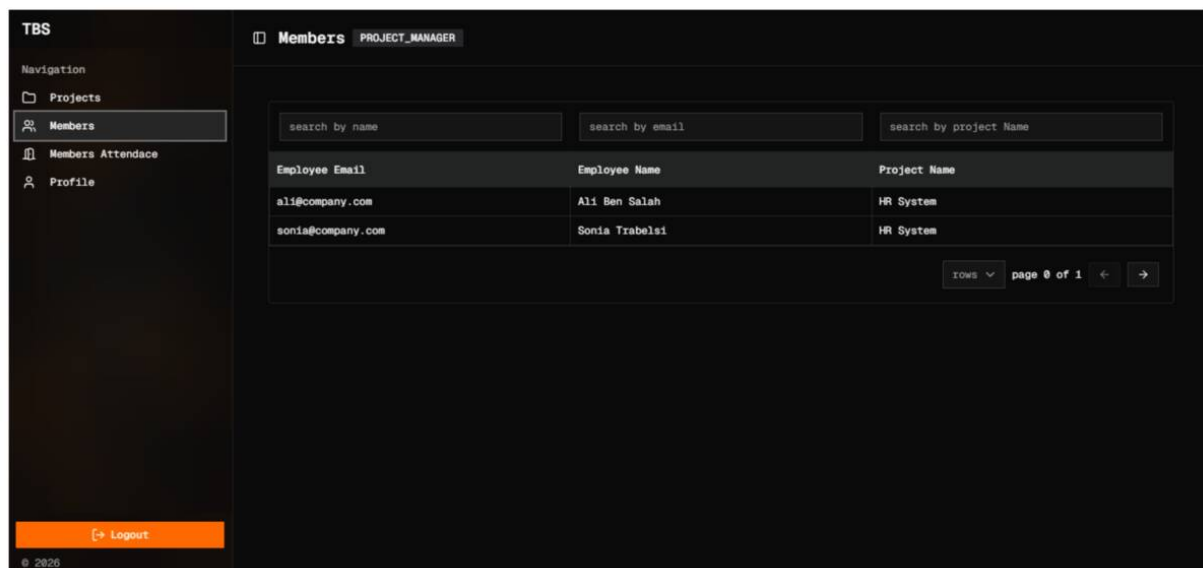


Figure 7.6: Project Manager Members Page

Figure 7.6 presents members linked to assigned projects for manager-level team supervision.

7.5.4 Members attendance monitoring interface

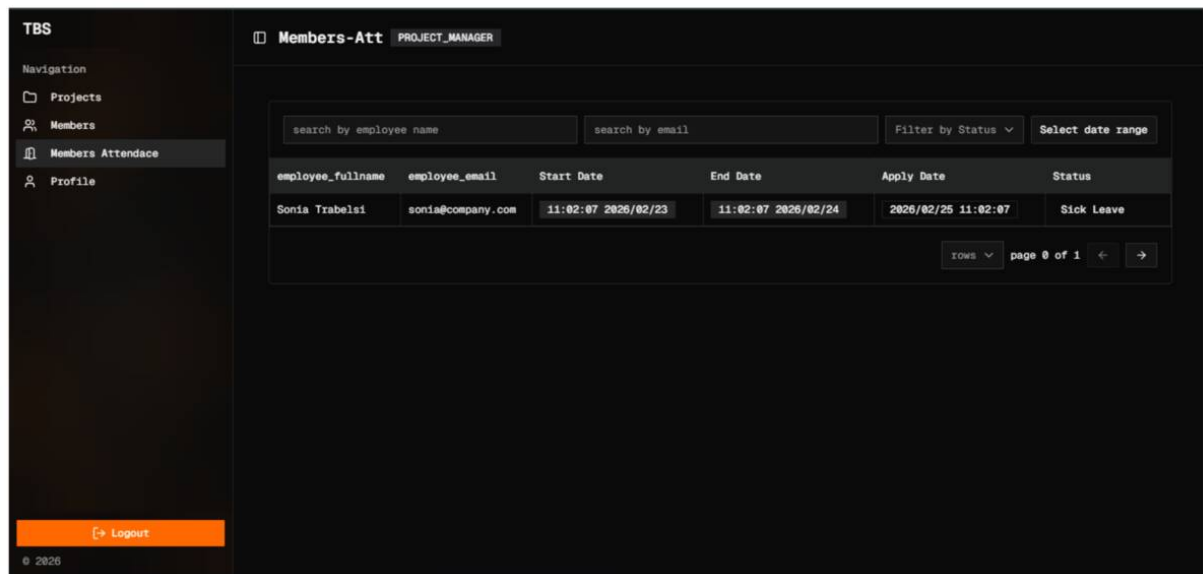


Figure 7.7: Project Manager Members and Attendance Monitoring

Figure 7.7 illustrates member-level supervision and attendance tracking within assigned projects.

7.6 Conclusion

The Project Manager modules delivered role-scoped supervision features and stable manager-facing workflows for project and attendance monitoring.

Chapter 8

RH Agent Integration

Contents

8.1	Introduction	71
8.2	Sprint Backlog	71
8.3	Sprint Design	71
8.3.1	Conversation and interaction design	71
8.3.2	LangGraph routing design	72
8.4	Conception	73
8.4.1	RH Agent sequence diagram	73
8.4.2	Agent state model conception	74
8.5	Sprint Realization	74
8.5.1	Implementation outcomes	74
8.5.2	RH Agent interface	75
8.6	Conclusion	75

8.1 Introduction

This chapter explains how we integrated the RH Agent in the final stage of the project. The goal was simple: help HR users through a conversational assistant that can answer questions, guide them, and also query data when needed.

8.2 Sprint Backlog

For this stage, the backlog focused on getting the RH Agent ready in a safe and usable way.

ID	Backlog Item	Priority
RH-BL-01	Integrate RH Agent capabilities into the platform workflow.	High
RH-BL-02	Connect RH Agent interactions to authorized backend endpoints.	High
RH-BL-03	Validate response consistency and role-aware usage boundaries.	Medium
RH-BL-04	Execute final stabilization and end-to-end verification.	Medium

8.3 Sprint Design

The RH Agent was designed to be practical in real HR usage: clear replies, relevant context, and stable behavior during day-to-day operations.

8.3.1 Conversation and interaction design

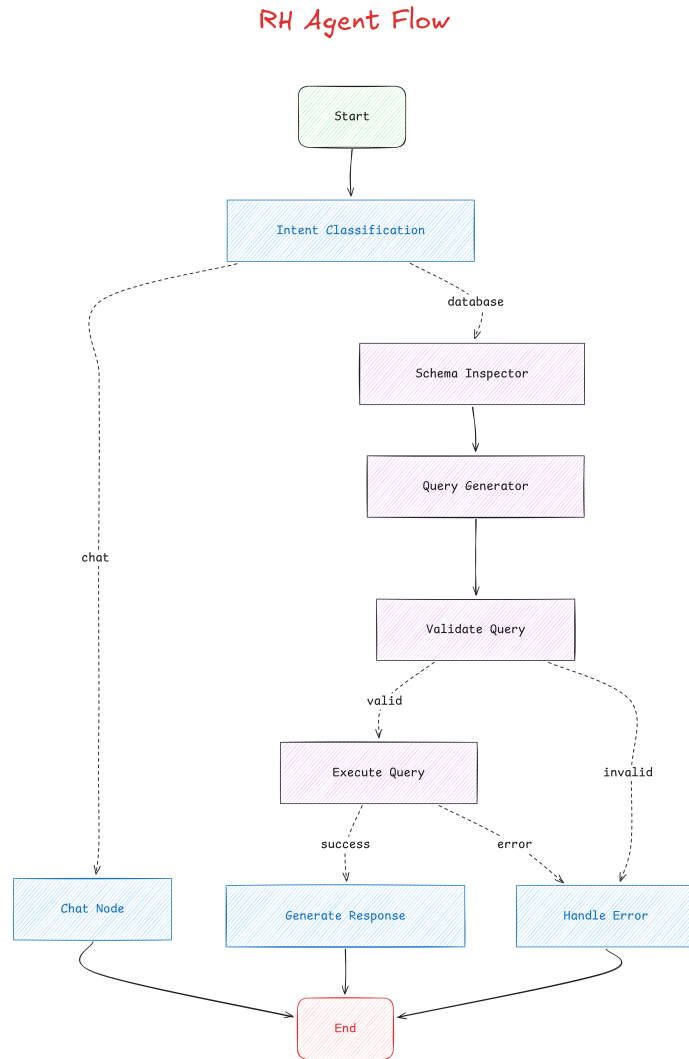
We first designed interaction flows for the most common HR cases: asking about attendance, checking follow-up actions, and helping users move to the right page quickly. We also kept responses short and useful instead of overly verbose.

8.3.2 LangGraph routing design

The RH Agent was built using **LangGraph** and **LangChain**, with **SQLAlchemy** for database access. The first thing the agent does is classify user input into two paths: a **chat** path and a **database-query** path. If the request is simple conversation, the agent replies directly. If it needs data, it goes through schema inspection, SQL generation, validation, execution, and finally response generation, with error handling at each important step.

Figure 8.1 summarizes the graph: input classification, chat node, database-query pipeline, and terminal success/error transitions.

Figure 8.1: RH Agent flow with intent classification and SQL routing

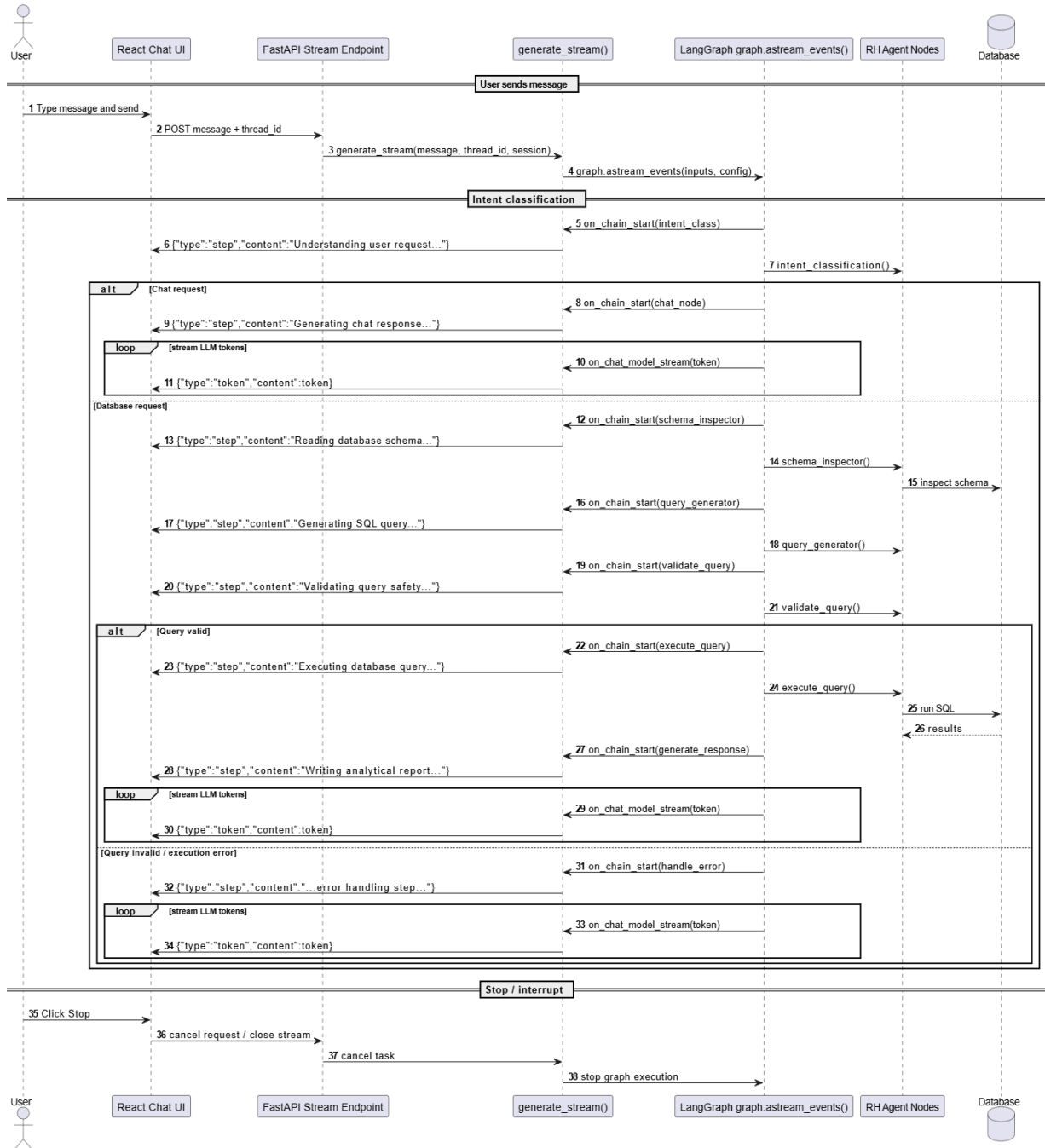


8.4 Conception

8.4.1 RH Agent sequence diagram

The sequence diagram shows the full journey from user request to final response: user interaction, intent classification, graph routing, SQL access when needed, and streamed output. Figure 8.2 summarizes request routing, query lifecycle, response streaming, and interruption handling through the stop action.

Figure 8.2: Sequence diagram of the RH Agent workflow



8.4.2 Agent state model conception

In LangGraph, the **agent state** is the shared memory used by all nodes while a request is being processed. You can see it as one object that moves across the flow and keeps the key information needed at each step. We designed it to support both normal chat answers and database-query answers.

The state contains:

- **db_context:** `str`: short database context (schema/table information) used to help the LLM generate relevant SQL.
- **messages:** `list[BaseMessage]`: conversation history between the user and the assistant.
- **user_input:** `str`: the latest user message to process.
- **intent:** `Optional[str]`: detected route, typically `chat` or `database-query`.
- **sql_query:** `Optional[str]`: SQL generated by the query node before execution.
- **sql_is_safe:** `bool`: safety flag showing whether the SQL passed validation checks.
- **query_result:** `str`: formatted database output returned to the user.
- **error:** `Optional[str]`: error message captured when validation or execution fails.

For a bachelor-project perspective, this design is important because it separates responsibilities clearly: classification, SQL generation, validation, execution, and response formatting. Each step reads and updates only the fields it needs. This makes the agent easier to debug, safer for database access, and easier to maintain as new features are added.

8.5 Sprint Realization

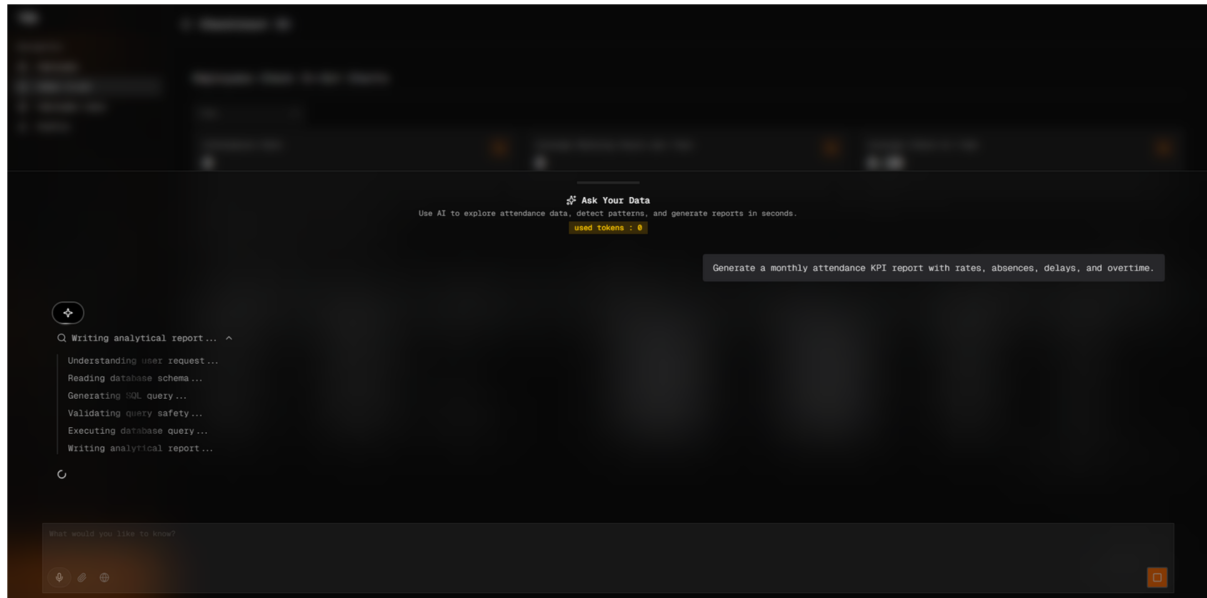
8.5.1 Implementation outcomes

- Integrated RH Agent module into the platform workflow.
- Connected RH Agent requests to selected backend services using LangChain and SQLAlchemy.
- Implemented intent classification to route requests between chat and database-query paths.
- Implemented schema inspection, SQL generation, query safety validation, execution, and error handling in the database route.
- Implemented streaming response delivery so users can read the agent output progressively during generation.
- Implemented a **Stop** control that allows users to interrupt generation in the middle of a conversation.
- Integrated support for local LLM execution (Ollama) with compatibility for cloud-hosted models.
- Performed stabilization and end-to-end verification.

8.5.2 RH Agent interface

Figure 8.3 presents the integrated assistant interface and a sample guided interaction flow. It highlights how users can type natural requests, receive streamed responses, and interrupt generation when needed.

Figure 8.3: RH Agent interface with an example of user interaction



8.6 Conclusion

This final stage made the platform more practical to use: users can ask naturally, get guided answers, stream results in real time, and stop generation whenever needed. Overall, the RH Agent improved both usability and operational efficiency.

References

Bibliography

- [1] OneTech Business Solutions, *Official Website*, available at: <https://www.onetech-bs.com/> (accessed April 6, 2026).
- [2] OneTech Business Solutions, *LinkedIn Company Page*, available at: <https://www.linkedin.com/company/onetech-business-solutions/> (accessed April 6, 2026).
- [3] Ken Schwaber and Jeff Sutherland, *The Scrum Guide: The Definitive Guide to Scrum (November 2020)*, available at: <https://scrumguides.org/scrum-guide.html> (accessed April 6, 2026).
- [4] Ken Schwaber and Jeff Sutherland, *The Scrum Guide (PDF, November 2020)*, available at: <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf> (accessed April 6, 2026).
- [5] Scrum.org, *What is Scrum?*, available at: <https://www.scrum.org/resources/what-is-scrum> (accessed April 6, 2026).
- [6] Scrum Alliance, *What is Scrum? A Guide to the Scrum Framework*, available at: <https://resources.scrumalliance.org/Article/what-is-scrum> (accessed April 6, 2026).
- [7] Beck, K. et al., *Manifesto for Agile Software Development*, available at: <https://agilemanifesto.org/> (accessed April 6, 2026).
- [8] LangChain, *LangGraph Overview*, available at: <https://docs.langchain.com/oss/python/langgraph/overview> (accessed April 7, 2026).
- [9] LangChain, *LangChain Overview*, available at: <https://docs.langchain.com/oss/python/langchain/overview> (accessed April 7, 2026).
- [10] SQLAlchemy, *SQLAlchemy ORM Documentation*, available at: <https://docs.sqlalchemy.org/en/20/orm/> (accessed April 7, 2026).
- [11] Ollama, *Ollama Documentation*, available at: <https://docs.ollama.com/> (accessed April 7, 2026).
- [12] Redis, *Redis Documentation*, available at: <https://redis.io/docs/latest/> (accessed April 21, 2026).
- [13] FastAPI, *FastAPI Documentation*, available at: <https://fastapi.tiangolo.com/> (accessed April 7, 2026).

- [14] FastAPI, *FastAPI Features*, available at: <https://fastapi.tiangolo.com/features/> (accessed April 7, 2026).
- [15] Alembic, *Alembic Documentation*, available at: <https://alembic.sqlalchemy.org/en/latest/> (accessed April 7, 2026).
- [16] PostgreSQL Global Development Group, *PostgreSQL Documentation*, available at: <https://www.postgresql.org/docs/> (accessed April 7, 2026).
- [17] Oracle, *MySQL Documentation*, available at: <https://dev.mysql.com/doc/> (accessed April 7, 2026).
- [18] Resend, *Resend Documentation*, available at: <https://resend.com/docs> (accessed April 7, 2026).
- [19] Docker, *What is Docker?*, available at: <https://docs.docker.com/engine/docker-overview/> (accessed April 7, 2026).
- [20] React, *React Documentation*, available at: <https://react.dev/> (accessed April 7, 2026).
- [21] Tailwind Labs, *Tailwind CSS*, available at: <https://tailwindcss.com/> (accessed April 7, 2026).
- [22] shadcn, *shadcn/ui Documentation*, available at: <https://ui.shadcn.com/docs> (accessed April 7, 2026).
- [23] TanStack, *TanStack Table Documentation*, available at: <https://tanstack.com/table/latest/docs> (accessed April 7, 2026).
- [24] TanStack, *TanStack Query Documentation*, available at: <https://tanstack.com/query/latest/> (accessed April 7, 2026).
- [25] Git, *Git Documentation*, available at: <https://git-scm.com/docs/git> (accessed April 7, 2026).
- [26] GitHub, *GitHub Docs*, available at: <https://docs.github.com/> (accessed April 7, 2026).
- [27] Microsoft, *Visual Studio Code Documentation*, available at: <https://code.visualstudio.com/docs> (accessed April 7, 2026).
- [28] JetBrains, *PyCharm Documentation*, available at: <https://www.jetbrains.com/help/pycharm/essentials.html> (accessed April 7, 2026).